



МИНОБРНАУКИ РОССИИ
федеральное государственное автономное образовательное учреждение
высшего образования
«Московский государственный технологический университет «СТАНКИН»
(ФГАОУ ВО «МГТУ «СТАНКИН»)

**Институт
информационных
технологий**

**Кафедра
информационных
технологий и
вычислительных систем**

Кочановский Кирилл Андреевич

Выпускная квалификационная работа
по направлению подготовки
09.04.04 «Программная инженерия»

Профиль: «Технологии разработки интеллектуальных систем и программных комплексов»

**ИССЛЕДОВАНИЕ МЕТОДОВ
АВТОМАТИЧЕСКОГО СОСТАВЛЕНИЯ
РАСПИСАНИЯ И РАЗРАБОТКА ПРОГРАММНОГО
СРЕДСТВА ПОДДЕРЖКИ ГЕНЕРАЦИИ
РАСПИСАНИЯ ДЛЯ МГТУ «СТАНКИН»**

Регистрационный номер № _____

Зав. кафедрой	_____	к.т.н., доц. Новоселова О. В.
Научный руководитель	_____	к.т.н. Гаврилов А. Г.
Научный консультант	_____	ст. преподаватель Тохунц Н. Б.
Обучающийся	_____	Кочановский К. А.

Москва 2026 г.



МИНОБРНАУКИ РОССИИ
федеральное государственное автономное образовательное учреждение
высшего образования
«Московский государственный технологический университет «СТАНКИН»
(ФГАОУ ВО «МГТУ «СТАНКИН»)

**Институт
информационных
технологий**

**Кафедра
информационных
технологий и
вычислительных систем**

«УТВЕРЖДАЮ»
Заведующий кафедрой ИТиВС
к.т.н., доц. Новоселова О. В.

«___» _____ 2026 г.

Задание

на выполнение выпускной квалификационной работы
по направлению подготовки
09.04.04 «Программная инженерия»
Профиль: «Технологии разработки интеллектуальных систем и программных комплексов»

Студент группы ИДМ-24-08
Кочановский К. А.

Научный руководитель
доцент, к.т.н. Гаврилов А. Г.

**Тема: «Исследование методов автоматического составления
расписания и разработка программного средства поддержки
генерации расписания для МГТУ «СТАНКИН»**

Тема утверждена приказом «___». _____ 202_ г. № ____
Срок сдачи ВКР на кафедру «___». _____ 202_ г.

1. Описание задания на выполнение ВКР

1.1. Тип ВКР — исследовательская работа.

1.2. Цель исследования — повышение эффективности деятельности учебного заведения.

1.3. Объект исследования — программная поддержка процесса создания расписания учебных занятий.

1.4. Предмет исследования — архитектура и функционирование клиент-серверного веб-приложения для создания расписания учебных занятий с учётом личных предпочтений преподавательского состава.

1.5. Методы исследования — системный анализ, процессно-ориентированный подход и функциональное моделирование процессов, объектно-ориентированный анализ.

1.6. Задачи исследования:

1.6.1. Провести исследование различных методов автоматического составления расписаний учебных занятий. Сделать обоснованные выводы о наличии необходимости в разработке своего метода. Провести анализ правил составления расписания МГТУ «СТАНКИН».

1.6.2. Обосновать требования для клиент-серверного веб-приложения для создания расписания учебных занятий.

1.6.3. Разработать проект клиент-серверного веб-приложения для создания расписания учебных занятий.

1.6.4. Разработать клиент-серверное веб-приложение для создания расписания учебных занятий в соответствии с разработанным проектом.

2. Требования к выполнению ВКР

2.1. Соблюдение требований законодательной базы и стандартов

2.1.1. Образовательная программа высшего образования в магистратуре ФГБОУ ВО «МГТУ «СТАНКИН» по направлению подготовки 09.04.04 «Программная инженерия» для направленности (профиля) подготовки «Технологии разработки интеллектуальных систем и программных комплексов» (утв. 31.05.2021).

2.1.2. П 01-04/7/2024. Положение о государственной итоговой аттестации по образовательным программам высшего образования — программам бакалавриата, программам специалитета и программам магистратуры —

утверждено приказом ректора от 03.04.2024 г. №700/1. — ФГБОУ ВО «МГТУ «СТАНКИН».

2.1.3. П 01-04/9/2024. Положение о выпускной квалификационной работе обучающихся по образовательным программам высшего образования — программам бакалавриата, программам специалитета, программам магистратуры — утверждено приказом ректора от 02.09.2024. №700/1. — ФГБОУ ВО «МГТУ «СТАНКИН».

2.2. Дополнительные требования

2.2.1. Оформление раздела «Список литературы» по национальному стандарту ГОСТ Р 7.0.100-2018 «Библиографическая запись. Библиографическое описание. Общие требования и правила составления»

2.2.2. Результаты исследования должны быть опубликованы в виде научных статей и тезисов докладов (не менее 2), а также доложены на научно-технических конференциях и семинарах.

3. Срок сдачи оформленной квалификационной работы на кафедру — вторая декада мая 2026 г.

Исполнитель

Кочановский К. А.

Научный руководитель

доцент каф. ИТиВС, к.т.н.

Гаврилов А. Г.

АННОТАЦИЯ

выпускной квалификационной работы
студента группы ИДМ-24-08 ФГАОУ ВО «МГТУ «СТАНКИН»

Кочановского Кирилла Андреевича

по направлению подготовки

09.04.04«Программная инженерия»

на тему:

**«Исследование методов автоматического составления расписания и
разработка программного средства поддержки генерации расписания
для МГТУ «СТАНКИН»»**

Актуальность работы. Формирование расписания учебных занятий является одной из основных и наиболее сложных задач автоматизации управления учебным процессом любого учебного заведения. МГТУ «СТАНКИН» не является исключением, каждый семестр специальный отдел в составе Учебно-методического управления МГТУ «СТАНКИН» [1] составляет расписание на следующие полгода, выкладывает его на сайт университета и отправляет преподавателям и старостам групп. На сегодняшний день процесс создания расписания учебных занятий производится вручную, что является медленным и ресурсозатратным процессом. В данной выпускной квалификационной работе будет проведён анализ существующих подходов к решению данной задачи, а также предложен новый алгоритм решения. Финальным шагом работы будет разработка клиент-серверного веб-приложения для создания расписания университета.

Научная новизна работы состоит в предложении гибридного метаэвристического подхода к составлению расписания вуза: алгоритм коллапса волновой функции (WFC) формирует допустимое по жёстким ограничениям расписание в модели «группа × день × слот» с настраиваемыми весами и недельным распространением занятий,

эволюционная оптимизация с repair-мутациями и локальным поиском улучшает мягкие критерии (окна, равномерность нагрузки, недельный паттерн), назначение аудиторий выполняется отдельным этапом с учётом «компоновки вуза». Комбинация WFC и эволюционной схемы для задачи university timetabling в описанной постановке ранее в литературе не рассматривалась.

Практическая ценность работы состоит в автоматизации такого ресурсозатратного процесса как планирование и формирование плана учебных занятий на семестр.

Структура работы. Выпускная квалификационная работа (ВКР) содержит 76 стр. сквозной нумерации. Количество рисунков в работе — 14, таблиц — 7. Позиций в списке литературы — 35.

Выводы:

1. Проведён анализ методов автоматического составления расписания и обоснован выбор гибридного подхода — WFC для первичного допустимого решения и эволюционная оптимизация на основе генетического алгоритма для его улучшения.
2. Разработаны требования к расписанию, описан предложенный алгоритм генерации и оптимизации, спроектирована архитектура и стек технологий клиент-серверного приложения и реализовано программное средство с веб-интерфейсом.
3. Выполнено экономическое обоснование целесообразности внедрения разработанного средства для МГТУ «СТАНКИН».

Список работ, опубликованных по теме ВКР:

1. Работа, опубликованная на студенческой научно-практической конференции «Автоматизация и информационные технологии (АИТ-2024)» МГТУ «СТАНКИН» — «Исследование методов автоматического составления расписания» [2].
2. Работа, опубликованная на 68-й Всероссийской научной конференции МФТИ — «Алгоритм решения задач составления расписания занятий

в образовательных учреждениях» [3].

ОГЛАВЛЕНИЕ

ПЕРЕЧЕНЬ СОКРАЩЕНИЙ И СПЕЦИАЛЬНЫХ ТЕРМИНОВ	10
ВВЕДЕНИЕ	11
ГЛАВА 1. Анализ предметной области	14
1.1. Анализ методов автоматического создания расписания учебных занятий.....	14
1.1.1. Линейные и целочисленные программные методы составления расписания	15
1.1.2. Метаэвристики	16
1.1.3. Гиперэвристики	18
1.2. Выводы по первой главе.....	19
ГЛАВА 2. Разработка алгоритма генерации расписания.....	21
2.1. Минимальные требования к расписанию	21
2.2. Допустимое решение	23
2.2.1. Коллапс волновой функции	23
2.2.2. Satisfiability solvers	27
2.3. Оценка решения.....	29
2.4. Оптимизация решения	30
2.4.1. Генетический алгоритм	30
2.4.2. Имитация отжига	31
2.5. Вывод по второй главе	32
ГЛАВА 3. Разработка архитектуры сервиса для программной поддержки генерации расписания учебных заведений	33
3.1. Подход к созданию программного решения	33
3.2. Формирование архитектуры базы данных	33
3.2.1. ERD диаграмма базы данных.....	34
3.3. Вывод по третьей главе	35
ГЛАВА 4. Выбор средств реализации сервиса для программной поддержки генерации расписания учебных заведений	36

4.1. Обоснование выбора СУБД.....	36
4.1.1. Хранение данных в текстовых файлах.....	37
4.1.2. SQLite	37
4.1.3. Вывод	38
4.2. Выбор вычислительного движка	38
4.3. Выбор средств реализации сервера	39
4.4. Выбор средств реализации клиента.....	39
4.5. Вывод по четвёртой главе	40
ГЛАВА 5. Реализация сервиса для программной поддержки генерации расписания учебных заведений.....	42
5.1. Заполнение базы данных	42
5.1.1. Процесс популяции базы данных	42
5.2. Сервер	43
5.2.1. Реализация WFC на языке Polars.....	44
5.2.2. Реализация генетического алгоритма на языке Polars	50
5.3. Экспериментальная оценка алгоритма	55
5.3.1. Назначение аудиторий	57
5.4. Клиент	59
5.5. Вывод по пятой главе	60
ГЛАВА 6. Экономическое обоснование	62
6.1. Анализ аналогов и обоснование собственного решения	62
6.2. Оценка затрат проекта	65
6.3. Обоснование экономической выгоды проекта.....	68
6.4. Вывод по экономическому обоснованию	70
ЗАКЛЮЧЕНИЕ	71
СПИСОК ЛИТЕРАТУРЫ.....	73

ПЕРЕЧЕНЬ СОКРАЩЕНИЙ И СПЕЦИАЛЬНЫХ ТЕРМИНОВ

Сокращения:

API (Application Programming Interface) — программный интерфейс приложения

ERD (Entity-Relationship Diagram) — модель данных, позволяющая описывать концептуальные схемы предметной области

ERP (Enterprise Resource Planning) — планирование ресурсов предприятия

PWA (Progressive Web Application) — прогрессивное веб-приложение

REST (Representational State Transfer) — архитектурный стиль взаимодействия компонентов распределённого приложения в сети

SA (Simulated Annealing) — имитация отжига

SAT (Boolean Satisfiability Problem) — задача булевой выполнимости

SMT (Satisfiability Modulo Theories) — выполнимость по моделям теорий

UI (User Interface) — пользовательский интерфейс

WFC (Wave Function Collapse) — коллапс волновой функции

СУБД (Система управления базами данных)

УМУ (Учебно-методическое управление)

ЭОС (Электронная образовательная среда)

Термины:

Парсинг — автоматизированный сбор и структурирование информации

Фреймворк структурированный набор инструментов и библиотек, облегчающий разработку программного обеспечения

ВВЕДЕНИЕ

Формирование расписания учебных занятий является одной из основных и наиболее сложных задач автоматизации управления учебным процессом вуза. Каждый семестр специальный отдел в составе Учебно-методического управления МГТУ «СТАНКИН» [1] составляет расписание на следующие полгода, выкладывает его на сайт университета и отправляет преподавателям и старостам групп. На сегодняшний день процесс создания расписания учебных занятий производится вручную, что является медленным и ресурсозатратным действием.

Целью выпускной квалификационной работы является повышение эффективности деятельности учебного заведения за счёт исследования методов автоматической генерации расписания и разработки программного средства, реализующего такую генерацию с учётом регламента вуза, предпочтений преподавателей и «компоновки вуза».

Объектом исследования является программная поддержка процесса создания расписания учебных занятий.

Предметом исследования являются архитектура и функционирование клиент-серверного веб-приложения для создания расписания учебных занятий с учётом личных предпочтений преподавательского состава.

Для достижения цели поставлены следующие задачи:

1. Провести исследование методов автоматического составления расписаний учебных занятий, обосновать необходимость собственного подхода и проанализировать правила составления расписания МГТУ «СТАНКИН».
2. Сформулировать требования к клиент-серверному веб-приложению для создания расписания учебных занятий.
3. Разработать проект клиент-серверного веб-приложения.
4. Реализовать клиент-серверное веб-приложение в соответствии с разработанным проектом.

Методы исследования: системный анализ, процессно-ориентированный подход и функциональное моделирование процессов, объектно-ориентированный анализ, сравнительный анализ алгоритмов и программная реализация предложенного решения.

В работе будут рассмотрены методы автоматического создания расписания, предложен гибридный алгоритм на основе коллапса волновой функции (WFC) и эволюционной оптимизации, а также реализован сервис, выполняющий следующие функции:

1. Генерация расписания с учётом учебных планов, нормативов, списков преподавателей, групп и т. д.
2. Настройка распределения занятий по неделе и в течение дня, в том числе формирование разгрузочных дней и смещение учебного процесса в пределах суток.
3. «Компоновка вуза» для сокращения времени на переходы между учебными занятиями.
4. Вывод расписания в различных форматах, включая веб-интерфейс для просмотра преподавателями и студентами.

Так как качество расписания определяет эффективность учебного процесса, значительное внимание уделено оценке расписаний и их последующему улучшению.

Научная новизна заключается в адаптации WFC к модели «группа × день × слот» с учётом регламента вуза и в гибридной схеме оптимизации: построение допустимого по жёстким ограничениям расписания с последующим улучшением эволюционным методом с *gerair*-мутациями и отдельным этапом назначения аудиторий.

Практическая ценность состоит в автоматизации ресурсозатратного процесса планирования учебных занятий на семестр и в снижении числа нарушений нормативов при пересборке расписания.

Структура работы. Первая глава содержит обзор методов решения задачи. Во второй главе сформулированы требования к расписанию и описан

предложенный алгоритм. Третья и четвёртая главы посвящены архитектуре и выбору средств реализации. Пятая глава описывает программную реализацию и результаты экспериментов. Экономическое обоснование приведено в отдельной главе. В заключении подведены итоги и намечены направления дальнейшей работы.

ГЛАВА 1. АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ

1.1. АНАЛИЗ МЕТОДОВ АВТОМАТИЧЕСКОГО СОЗДАНИЯ РАСПИСАНИЯ УЧЕБНЫХ ЗАНЯТИЙ

Задача создания расписания сводится к формированию и оптимизации процесса обслуживания конечного множества требований (заявок) на осуществление действий в системе. Задача составления расписания учебного заведения относится к классу NP-hard задач, она сводится к распределению трёх ресурсов: преподавательского состава, студенческого состава и аудиторий учебного заведения. Детальная постановка задачи, требования к расписанию и предлагаемый алгоритм приводятся во второй главе? в настоящей главе рассматриваются существующие методы решения и обосновывается выбор гибридного подхода.

Если посмотреть на количество научных статей (см. Рис. 1.1), публикуемых международным сообществом на эту тему за последние пару десятков лет [4], то можно сделать вывод, что область образовательной логистики всё ещё является актуальной темой исследований и разработки.

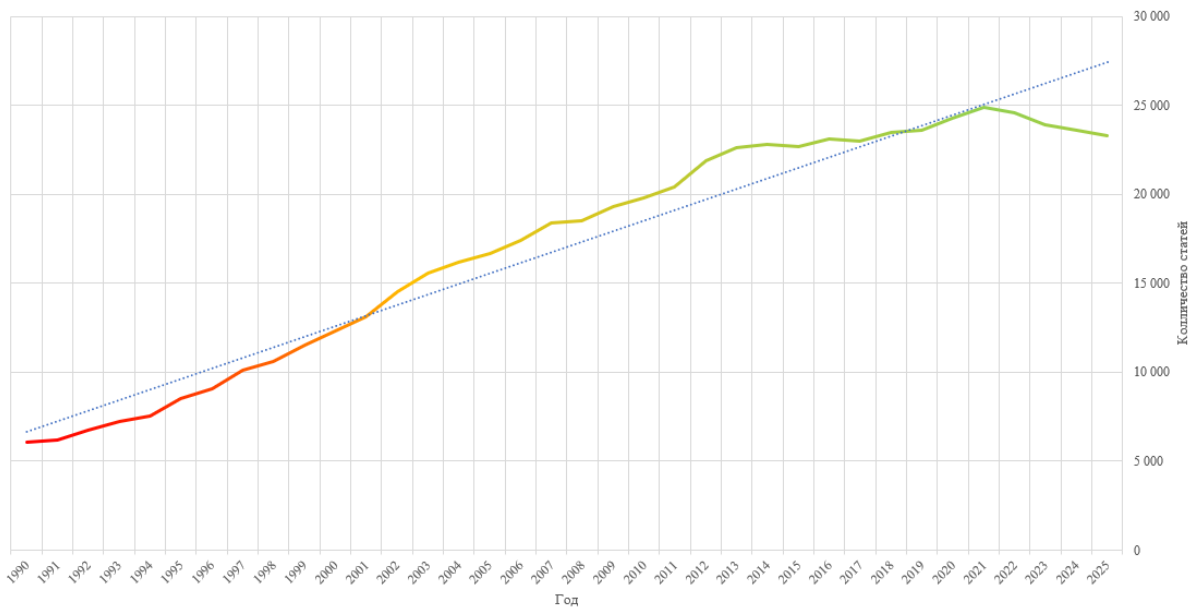


Рис. 1.1. Количество статей по годам на тему расписаний в высших учебных заведениях в Google Scholar по поисковому запросу «(timetabling OR timetable) AND higher education»

Для того, чтобы разобраться на каком этапе сейчас эта область, стоит проанализировать различные подходы и выявить основные тенденции и идеи.

1.1.1. Линейные и целочисленные программные методы составления расписания

Одними из первых, кто предоставил стратегии раскраски графов для решения проблемы составления расписания были Уэлш и Пауэлл (1967) [5]. Они заложили основу для более сложных исследований графовых эвристик в составлении расписания [6]. Эвристики составления расписания на основе раскраски графов - это полезный метод, на котором строится их оценка и улучшение.

Линейные и целочисленные программные методы - это математические алгоритмы, которые присваивают целочисленные значения переменным. Вариации этого метода были и до сих пор широко используются для решения комбинаторных задач оптимизации. Эти методы включают в себя программирование с ограничениями и методы удовлетворения

ограничений.

Однако такие методы, как правило, требуют больших вычислительных ресурсов из-за экспоненциального количества переменных.

1.1.2. Метаэвристики

Метаэвристика — это метод оптимизации, многократно использующий простые правила или эвристики для достижения оптимального или субоптимального решения. С 1995 по 2010 годы методологические подходы к решению проблем составления расписания обычно классифицировались на две категории метаэвристик: популяционные и одиночные [7]. Если классифицировать основные подходы и алгоритмы, то можно построить следующий график (см. Рис. 1.2).

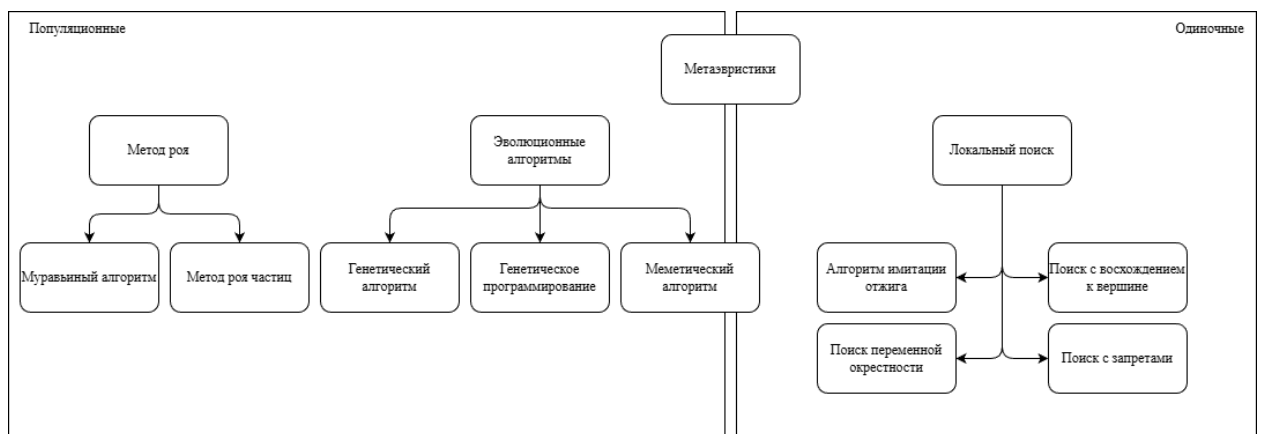


Рис. 1.2. Классификация алгоритмов решения задачи

Метаэвристический подход позволяет быстро получить приемлемое решение популяционным алгоритмом, а впоследствии улучшать его и удовлетворять дополнительные ограничения итеративным образом с помощью одиночных методов локального поиска.

Естественно нет возможности окунуться с деталями в каждый подход, но стоит хотя бы поверхностно изучить и оценить разные метаэвристические методы.

1.1.2.1. Локальные алгоритмы

Поиск с запретами. Поиск с запретами (Tabu search) относится к методологиям локального поиска и основан на поиске градиентного спуска. Существует несколько актуальных работ, в которых проведено ценное исследование методов поиска с запретами [8, 9]. Исследования основаны на диверсификации соседей, при котором поиск расширяется для нахождения большего количества локальных оптимумов. Это приводит к интенсификации шагов и более быстрого нахождения решения.

Однако поиск с запретами имеет тенденцию застрять в подоптимальных областях или плато. Подоптимальные области - это регионы пространства поиска, где текущее решение локально оптимально, но не обязательно глобально. Плато представляют собой плоские области функции, где градиент близок к нулю, что затрудняет продвижение поиска.

Алгоритм имитации отжига. Алгоритм SA — это ещё один метод локального поиска. SA вдохновлён физическим процессом отжига металла, при котором нагреваемый металл постепенно остывает, обеспечивая более устойчивую кристаллическую структуру. В контексте оптимизации, этот метод направлен на поиск более широкой области пространства поиска, в которой принимаются худшие шаги и допускается более широкий поиск оптимального решения. Существует множество вариаций SA [7, 10], он часто комбинируется с алгоритмом восхождения к вершине [11], или программированием с ограничениями [12].

Идеи стохастического локального поиска, заложенные в SA, используются в предлагаемом алгоритме на этапе оптимизации расписания при реализации repair-мутаций и локального поиска по целевой функции.

1.1.2.2. Эволюционные алгоритмы

Генетические алгоритмы. Среди эволюционных алгоритмов одними из самых изученных и популярных являются генетические алгоритмы (Genetic

algorithms) [13]. Эти алгоритмы моделируют процесс естественного отбора, таким образом находя более совершенную форму «жизни».

Меметические алгоритмы. Как дополнение к генетическим алгоритмам рассматриваются меметические алгоритмы [14]. Меметические алгоритмы (Memetic algorithms) представляют собой эволюционный метод оптимизации, который объединяет генетические алгоритмы с понятием «мема» из теории эволюции Ричарда Докинза. В данном контексте мем — это небольшая единица знания или идеи, которая может передаваться от одного индивида к другому.

Муравьиный алгоритм. Другой метод, основанный на популяции и более подробно исследованный в последнее десятилетие — это группа муравьиных алгоритмов (Ant algorithms) [15]. Эти алгоритмы отслеживают информацию, собранную в процессе поиска, и затем используют эту информацию для создания новых решений на следующих этапах.

1.1.3. Гиперэвристики

Как для подходов, основанных на одиночных решениях, так и для подходов, основанных на популяции, характерны свои недостатки. В последнее время большинство исследователей в области составления расписаний сосредотачивались на локальных или одиночных решениях, а не на алгоритмах, основанных на популяции [16].

Развитием, вытекающим из стандартных локальных и населённых решений проблем составления расписаний, является применение гиперэвристик. Гиперэвристики представляют собой метод поиска, в котором несколько эвристик объединяются и адаптируются. Различие между метаэвристическими и гиперэвристическими заключается в том, что гиперэвристики стремятся найти общепринятую методологию, а не решать конкретный экземпляр проблемы.

1.2. ВЫВОДЫ ПО ПЕРВОЙ ГЛАВЕ

Был проведён анализ подходов к автоматическому составлению расписаний в высших учебных заведениях по обзорной литературе. Задача относится к классу NP-hard; точные методы (целочисленное программирование, SAT/SMT) применимы для проверки выполнимости, но масштабирование на полный семестр вуза требует эвристик.

Одиночные метаэвристики эффективны для тонкой настройки уже построенного расписания, но с трудом формируют с нуля допустимое решение при большом числе жёстких ограничений. Популяционные методы расширяют область поиска, однако прямое кодирование полного семестра в хромосому приводит к высокой вычислительной стоимости и частым нарушениям жёстких ограничений.

По результатам анализа для поставленной задачи обоснован гибридный метаэвристический подход, сочетающий этапы разной природы:

1. Конструктивный этап — алгоритм коллапса волновой функции (WFC) для получения расписания, удовлетворяющего жёстким ограничениям (учебный план, календарь, занятость групп и преподавателей);
2. Популяционно-локальный этап — эволюционная оптимизация с *repair*-мутациями, элитизмом и локальным поиском по целевой функции для улучшения мягких критериев (окна, равномерность, недельный паттерн);
3. Жадная постобработка — назначение аудиторий с учётом «компоновки вуза».

Такая схема относится к классу гибридных метаэвристик (по аналогии с меметическими алгоритмами): глобальный конструктивный поиск сочетается с локальным улучшением в окрестности допустимых решений. Гиперэвристический уровень в явном виде не реализуется: набор операторов мутации фиксирован, но выбирается случайно на каждой итерации.

Принято решение о разработке программного сервиса, реализующего

описанный конвейер. Детали алгоритма, функции оценки и ограничений приводятся во второй главе.

ГЛАВА 2. РАЗРАБОТКА АЛГОРИТМА ГЕНЕРАЦИИ РАСПИСАНИЯ

Для автоматического создания расписания будет использоваться метаэвристический подход, комбинирующий в себе сразу несколько алгоритмов.

Сперва будет получено первичное приемлемое решение — то есть решение, удовлетворяющее минимальным (жёстким) требованиям. Допустимым решением для поставленной задачи будет являться расписание, содержащее в себе все дисциплины согласно учебному плану и не нарушающее никакие нормативы.

Далее итеративным методом расписание будет оцениваться и улучшаться путём перестановки различных учебных занятий местами.

2.1. МИНИМАЛЬНЫЕ ТРЕБОВАНИЯ К РАСПИСАНИЮ

В этой главе будут рассмотрены строгие требования к расписанию работы учебного заведения. Расписание на семестр должно строго соответствовать учебному плану и количеству часов лекций, семинаров и лабораторных занятий. При этом нельзя просто «запихнуть» по 10 учебных занятий в каждый день, существуют нормативы времени по нагрузке как на преподавателей, так и на студентов. Также нельзя забывать и о рабочих часах самого учебного заведения, пары должны начинаться в строго отведённое под них время с 8:30 утра, по 22:50, без учёта выходных дней и праздников. Список нерабочих праздничных дней задаётся «Трудовым кодексом Российской Федерации» от 30.12.2001 № 197-ФЗ (ред. от 14.02.2024) [17]. Упрощённая диаграмма документов и таблиц управления учебным процессом приведена на рисунке ниже (см. Рис. 2.1).

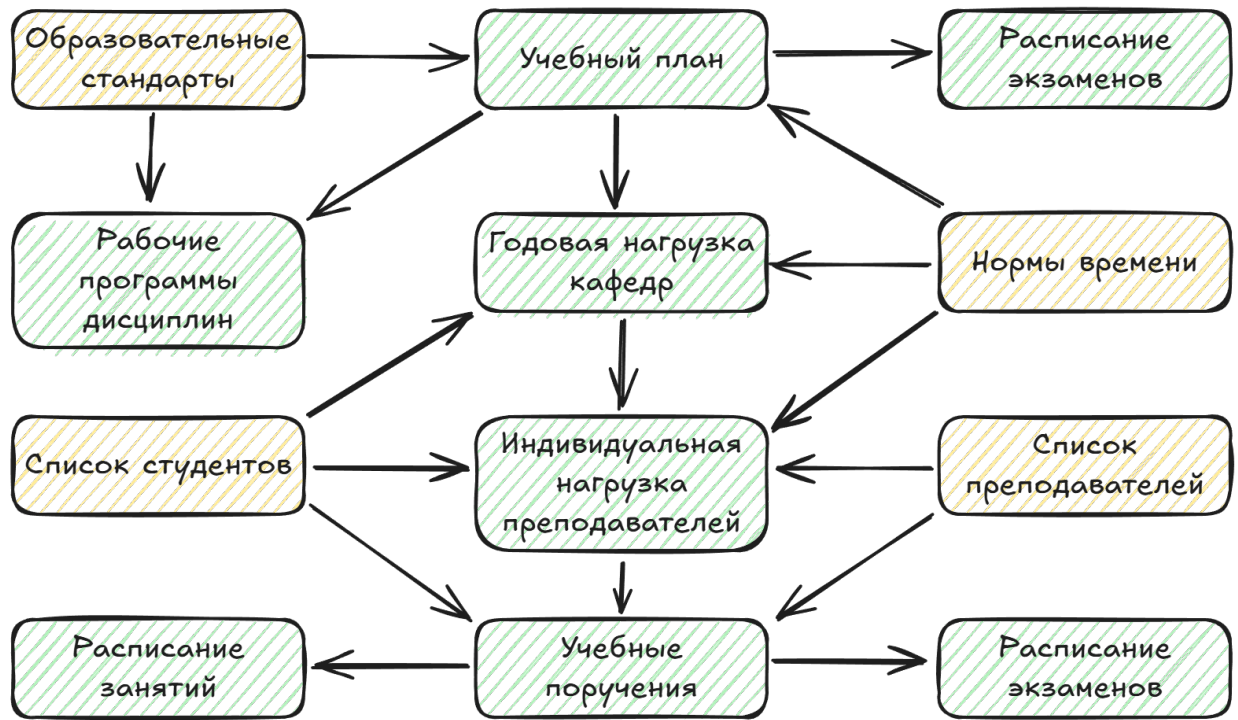


Рис. 2.1. Диаграмма документов и таблиц управления учебным процессом

Список студентов будет делиться на направления, согласно учебным планам, а также на группы и подгруппы для проведения семинаров и лабораторных занятий соответственно. Следует подметить, что деление на подгруппы является рудиментарным процессом, изначальный смысл которого заключался в невозможности вместить достаточное количество студентов в классы с компьютерами.

Не менее важным является подбор аудиторий для проведения учебных занятий. Главным параметром каждой аудитории является вместимость — нельзя назначать учебное занятие в аудиторию, которая физически не может вместить весь контингент студентов и преподавателей на данное учебное занятие. Дополнительным, не менее важным параметром является требуемое оборудование для проведения учебного занятия: мультимедийное и лабораторное оборудование.

Схематичное дробление контингента студентов и здания вуза предоставлено на рисунке ниже (см. Рис. 2.2).

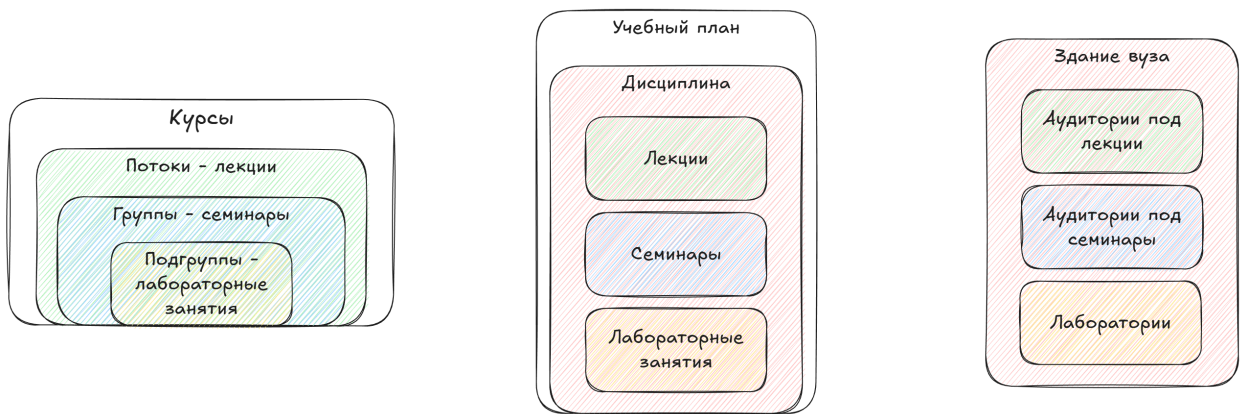


Рис. 2.2. Схематичное дробление контингента и здания вуза

2.2. ДОПУСТИМОЕ РЕШЕНИЕ

2.2.1. Коллапс волновой функции

WFC [18], вдохновлённый редукцией фон Неймана. Название алгоритма произошло от физического процесса, при котором волновая функция, описывающая вероятностное состояние квантовой системы, при измерении переходит из суперпозиции возможных состояний в одной конкретное наблюдаемое состояние.

В контексте задачи с составлением расписания её можно представить как суперпозицию здания университета. В каждой аудитории на каждый момент времени семестра существуют все возможные (удовлетворяющие ограничениям) учебные занятия конкретных групп и студентов. Для каждой такой ячейки высчитывается мера неопределённости — энтропия, то есть количество возможных состояний. Далее ячейки с наименьшей энтропией коллапсируют в конкретные состояния из набора возможных, при этом учитывается вес состояния. Весом выступает функция полезности, как например большие веса в начале дня для групп на дневном обучении, и наоборот для групп на вечернем обучении. Именно гибкость этой функции позволяет модифицировать алгоритм решения под конкретные особенности того или иного образовательного учреждения. Коллапс волновой функции позволит найти приемлемое решение, то есть то, которое соответствует

всем ограничениям, но в силу своей вероятностной природы решение не гарантирует глобального оптимума. Для того чтобы приблизить решение к глобальному оптимуму потребуется дополнительный шаг — генетический алгоритм.

2.2.1.1. Разрешение противоречий в WFC

Если в какой-то момент у всех доступных ячеек не остаётся допустимых вариантов коллапса, возникает противоречие.

Для решения таких ситуаций в работе рассматриваются четыре подхода:

- backtracking;
- backjumping;
- block based method;
- полный перезапуск с нуля.

Backtracking. Метод backtracking является классическим способом устранения противоречий в задачах поиска с ограничениями. Если после очередного выбора система приходит к неразрешимому состоянию, алгоритм возвращается к предыдущему шагу, отменяет последнее решение и пробует следующий вариант.

Формально процесс можно описать последовательностью решений формула (2.1):

$$S = \{s_1, s_2, \dots, s_k\}, \quad (2.1)$$

где S — последовательность принятых решений;

s_i — решение на шаге i ;

k — номер текущего шага.

Каждое решение влияет на множество допустимых значений для следующих ячеек. При обнаружении противоречия выполняется откат формула (2.2):

$$S \leftarrow \{s_1, s_2, \dots, s_{k-1}\}. \quad (2.2)$$

где $\leftarrow$ — возврат к предыдущему состоянию последовательности;

$k - 1$ — индекс шага, на который выполняется откат.

Преимущество *backtracking* заключается в том, что он позволяет сохранить уже построенную часть расписания и использовать её повторно. Это особенно важно в задачах, где большая часть размещённых занятий остаётся корректной, а конфликт возникает только в локальной области.

Однако у метода есть существенный недостаток: при большом числе ограничений количество возвратов может быстро расти. В задаче составления расписания это означает, что алгоритм может многократно пересчитывать одни и те же фрагменты, особенно если противоречие вызвано неудачным ранним выбором.

Backjumping. *Backjumping* является расширением *backtracking*. Вместо возврата к непосредственно предыдущему решению алгоритм анализирует причину противоречия и откатывается не на один шаг, а к наиболее раннему решению, которое действительно повлияло на возникновение ошибки.

Идея метода особенно полезна в расписаниях, где конфликт часто связан не с последним назначением, а с более ранним выбором, например:

- преподаватель уже оказался занят в нужное время;
- аудитория была закреплена за другим занятием;
- группа получила пересечение по нескольким предметам.

Если конфликт возник из-за решений s_i, s_j, s_k , то *backjumping* позволяет перейти не к s_{k-1} , а сразу к наиболее релевантному из них. Это сокращает число бесполезных откатов и уменьшает время поиска.

В практическом применении к расписанию метод *backjumping* полезен тогда, когда система хранит информацию о причинах исключения вариантов. Например, при удалении варианта можно фиксировать, какие уже назначенные занятия сделали его невозможным. Тогда при противоречии можно определить, какой именно шаг следует пересмотреть.

Недостаток метода состоит в том, что для его эффективной работы требуется дополнительный учёт причин конфликтов, а это усложняет

реализацию.

Block based method. Block based method основан на локализации проблемы внутри части пространства поиска. Вместо пересчёта всей модели алгоритм выделяет блок — набор связанных ячеек, внутри которого возникло противоречие, и пытается восстановить только этот фрагмент.

Для задачи расписания блоком может быть:

- одна учебная неделя;
- один день;
- конкретная пара групп, преподавателей и аудиторий.

Преимущество такого подхода состоит в том, что большая часть уже построенного расписания сохраняется. Это особенно удобно, если противоречие локализуется в ограниченной области и не затрагивает всю структуру решения.

Принцип работы можно описать следующим образом:

1. выделяется блок, в котором обнаружено противоречие;
2. внутри блока восстанавливается множество допустимых вариантов;
3. выполняется повторный коллапс только для этого блока;
4. если блок снова приводит к конфликту, он расширяется или пересобирается заново.

В рамках расписания такой подход уменьшает число глобальных перезапусков, но требует аккуратного определения границ блока. Если блок выбран слишком маленьким, противоречие может просто переместиться в соседнюю область; если слишком большим — метод теряет локальные преимущества.

Начать всё заново. Подход полного перезапуска предполагает, что при обнаружении тупика алгоритм сбрасывает текущее состояние и строит решение заново. В WFC это означает удаление всей текущей частичной раскладки и повторный запуск процесса коллапса с новыми случайными выборами.

Для задачи составления расписания такой подход особенно оправдан,

потому что качественное решение часто зависит от ранних случайных выборов. Если эти выборы оказались неудачными, дальнейшее локальное исправление может быть неэффективным. В этом случае проще и быстрее полностью перезапустить алгоритм и получить новую траекторию поиска.

Именно этот подход был выбран в реализации алгоритма WFC для генерации расписания ВУЗа, так как он является очень простым в реализации и на удивление эффективным.

2.2.2. Satisfiability solvers

Satisfiability solvers (SAT solvers) — это программы, решающие задачи булевой выполнимости (Boolean Satisfiability Problem).

Основным вопросом SAT-задачи является: «можно ли подобрать значения true/false для переменных так, чтобы логическое выражение стало истинным?»

Например, задана булева формула формула (2.3):

$$(A \vee B) \wedge (\neg A \vee C) \quad (2.3)$$

где A, B, C — булевы переменные.

SAT solver пытается найти такие значения переменных формула (2.4):

$$A = \text{false}, \quad B = \text{true}, \quad C = \text{true} \quad (2.4)$$

где false и true — логические значения «ложь» и «истина».

при которых вся формула становится истинной.

Если такого набора не существует, solver доказывает, что формула невыполнима.

2.2.2.1. Satisfiability Modulo Theories

Классический SAT solver работает только с булевой логикой, однако реальные задачи часто требуют работы с числами, массивами, строками и битовыми операциями.

Для этого используются SMT solver'ы (Satisfiability Modulo Theories) [19].

Например, SMT solver способен анализировать систему ограничений формула (2.5)–формула (2.7):

$$x > 10 \tag{2.5}$$

$$y = x + 2 \tag{2.6}$$

$$y < 5 \tag{2.7}$$

где x, y — целочисленные переменные;

10, 2, 5 — числовые константы в ограничениях.

и определить, что такая система противоречива.

2.2.2.2. Принцип работы

SAT/SMT solver — это механизм поиска значений, удовлетворяющих набору ограничений.

Вместо ручного перебора вариантов программист описывает:

- переменные;
- ограничения;
- условия задачи.

После этого solver самостоятельно ищет решение или доказывает, что решения не существует. Такая программа может быть полезной при формировании учебных планов, чтобы удостовериться, что план не будет нарушать жёсткие ограничения.

SMT-solver также может выступать в роли алгоритма формирования приемлемого решения, однако его природа заключается в том, чтобы найти

любое решение не быстро. WFC же в то время сможет не просто найти случайное решение, но и уже на этом этапе применить ряд оптимизаций решения.

2.3. ОЦЕНКА РЕШЕНИЯ

После получения первичного решения, которое удовлетворяет всем обязательным требованиям требуется провести итеративный процесс улучшения этого расписания. Для того, чтобы что-либо улучшать, нужно уметь давать оценку полученным результатам.

Самым важным критерием оценки является соответствие минимальным требованиям:

- все учебные занятия согласно учебному плану [20];
- весь учебный процесс проходит в период согласно календарному учебному графику [21];
- не нарушаются нормы времени;
- учебные занятия должны начинаться в строго отведённое под них время с 8:30 утра, по 22:50, без учёта выходных дней и праздников [17].

Для оценки будет рассчитываться множество различных параметров. Одним из самых главных таких параметров является непрерывность расписания — в прекрасном расписании будущего не должно быть «окон», на которых студенты имеют свойство расходиться по домам.

Ещё один важный параметр — это равномерность расписания. Не должно быть расписания, где в понедельник пять пар, а во вторник только одна. Согласно исследованию, проведённому Клеванским Николаем Николаевичем [22] студенты предпочитают, когда одни и те же пары проводятся в одно и то же время в разные дни недели. Соответственно существует три оценки равномерности:

- равномерность загруженности;
- равномерность начала занятий;
- равномерность идентичности пар по дням / неделям.

Дополнительным параметром может являться «компоновка» ВУЗа. Для экономии временных и финансовых ресурсов можно попробовать уместить весь учебный процесс в меньший набор аудиторий. Также можно пробовать уменьшать среднюю дистанцию, требуемую для перехода из одной аудитории в другую между учебными занятиями. В случае проведения лабораторных занятий дистанционно можно не делить на подгруппы, начинать семинары только после лекций по каждой дисциплине и т. д.

2.4. ОПТИМИЗАЦИЯ РЕШЕНИЯ

2.4.1. Генетический алгоритм

Генетический алгоритм представляет собой эвристический метод поиска оптимального решения, механизм работы которого аналогичен процессу естественного отбора в природе. В рамках данной задачи в качестве генома рассматривается полное расписание учебного заведения на семестр.

В научной литературе уже были упомянуты комбинации алгоритма WFC с генетическим алгоритмом [23], однако этот подход не применялся к задачам составления расписания.

Хромосома кодирует расписание на один семестр и состоит из набора генов, где каждый ген соответствует конкретному элементу расписания. Размер хромосомы определяется формула (2.8):

$$|H| = n \times 6 \times 182,625, \quad (2.8)$$

где $|H|$ — размер хромосомы;

n — количество учебных групп;

6 — количество учебных дней в неделю;

182,625 — усреднённое количество учебных дней в семестре (варьируется каждый семестр).

Операторы мутации реализуются с использованием локальных методов

поиска, например алгоритма имитации отжига, что позволяет эффективно исследовать окрестность текущего решения и избегать застревания в локальных минимумах.

2.4.2. Имитация отжига

Имитация отжига (SA) представляет собой стохастический метод локального поиска, основанный на аналогии с физическим процессом охлаждения твёрдых тел, при котором система постепенно переходит к состоянию с минимальной энергией. В контексте задачи оптимизации расписания данный алгоритм используется для улучшения текущего решения путём итерационного поиска его окрестностей (см. Рис. 2.3).

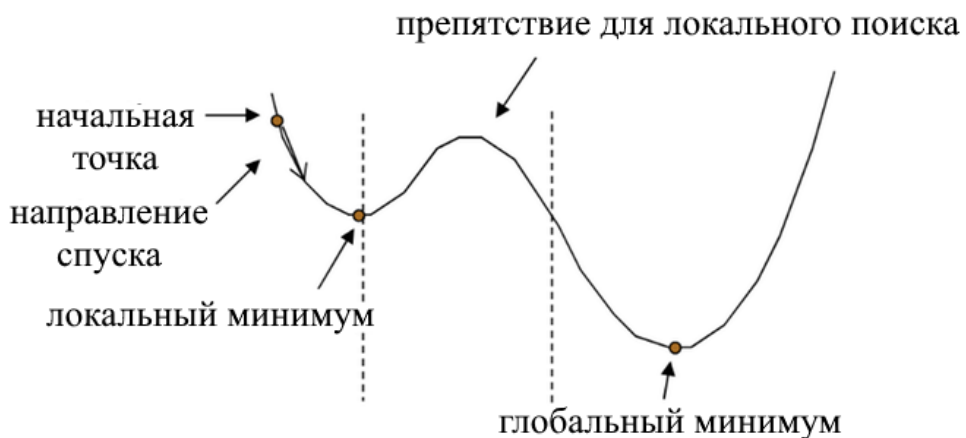


Рис. 2.3. Проблематика локальных методов поиска

На каждой итерации из текущего решения формируется новое, соседнее решение путём небольшого случайного изменения (например, перестановки занятий в расписании). Если новое решение имеет более высокое значение целевой функции, оно принимается. В противном случае оно может быть принято с некоторой вероятностью, определяемой формула (2.9):

$$P = \exp \left(-\frac{\Delta E}{T} \right), \quad (2.9)$$

где P — вероятность принятия ухудшающего решения;

T — параметр температуры алгоритма;

ΔE — ухудшение значения функции приспособленности.

Параметр температуры постепенно уменьшается согласно заданному расписанию охлаждения, что приводит к снижению вероятности принятия ухудшающих решений и обеспечивает сходимость алгоритма к локально оптимальному решению.

2.5. ВЫВОД ПО ВТОРОЙ ГЛАВЕ

Предложенный гибридный подход, основанный на применении алгоритма коллапса волновой функции и последующей оптимизации с помощью генетического алгоритма, позволяют решать задачу составления расписания, при этом поддерживая достаточную гибкость для адаптации к различным образовательным учреждениям.

ГЛАВА 3. РАЗРАБОТКА АРХИТЕКТУРЫ СЕРВИСА ДЛЯ ПРОГРАММНОЙ ПОДДЕРЖКИ ГЕНЕРАЦИИ РАСПИСАНИЯ УЧЕБНЫХ ЗАВЕДЕНИЙ

3.1. ПОДХОД К СОЗДАНИЮ ПРОГРАММНОГО РЕШЕНИЯ

Для разработки будущего сервиса была выбрана трёхзвенная архитектура (three-tier architecture):

1. База данных (data layer) — слой для хранения исходных данных, требуемых для генерации расписания, а также для хранения уже готовых расписаний.
2. Сервер (application layer) — на этом слое реализуется вся бизнес-логика и REST API эндпоинты для работы по протоколу HTTP(S).
3. Клиент (presentation layer) — финальный слой, необходимый для отображения интерфейса сервиса, для удобства взаимодействия с ним через браузер.

Подходом к написанию кода было выбрано объектно-ориентированное программирование (ООП). Таким образом все сущности, такие как расписание, учебная аудитория, преподаватель — становятся объектами, обладающими некими параметрами и методами. Это сильно упрощает разработку и инкапсулирует сложную логику внутри самих объектов.

3.2. ФОРМИРОВАНИЕ АРХИТЕКТУРЫ БАЗЫ ДАННЫХ

Прежде чем создавать таблицы базы данных и нормализировать их требуется определиться с данными, которые нужно хранить.

Есть четыре основные сущности:

- группа;
- преподаватель;
- аудитория;
- учебное занятие.

Каждая из сущностей должна содержать следующие данные:

1. Группа (group):

- имя группы;
- направления;
- институт;
- количество студентов.

2. Преподаватель (lecturer):

- институт;
- должность;
- ФИО.

3. Аудитория (classroom):

- номер аудитории;
- вместимость аудитории;
- оборудование.

4. Учебное занятие (subject):

- преподаватель;
- количество лекций / семинаров / лабораторных занятий;
- требуемое оборудование.

3.2.1. ERD диаграмма базы данных

Определив сущности и связи между ними можно построить ERD диаграмму архитектуры базы данных и привести её к третьей нормальной форме (см. Рис. 3.1).

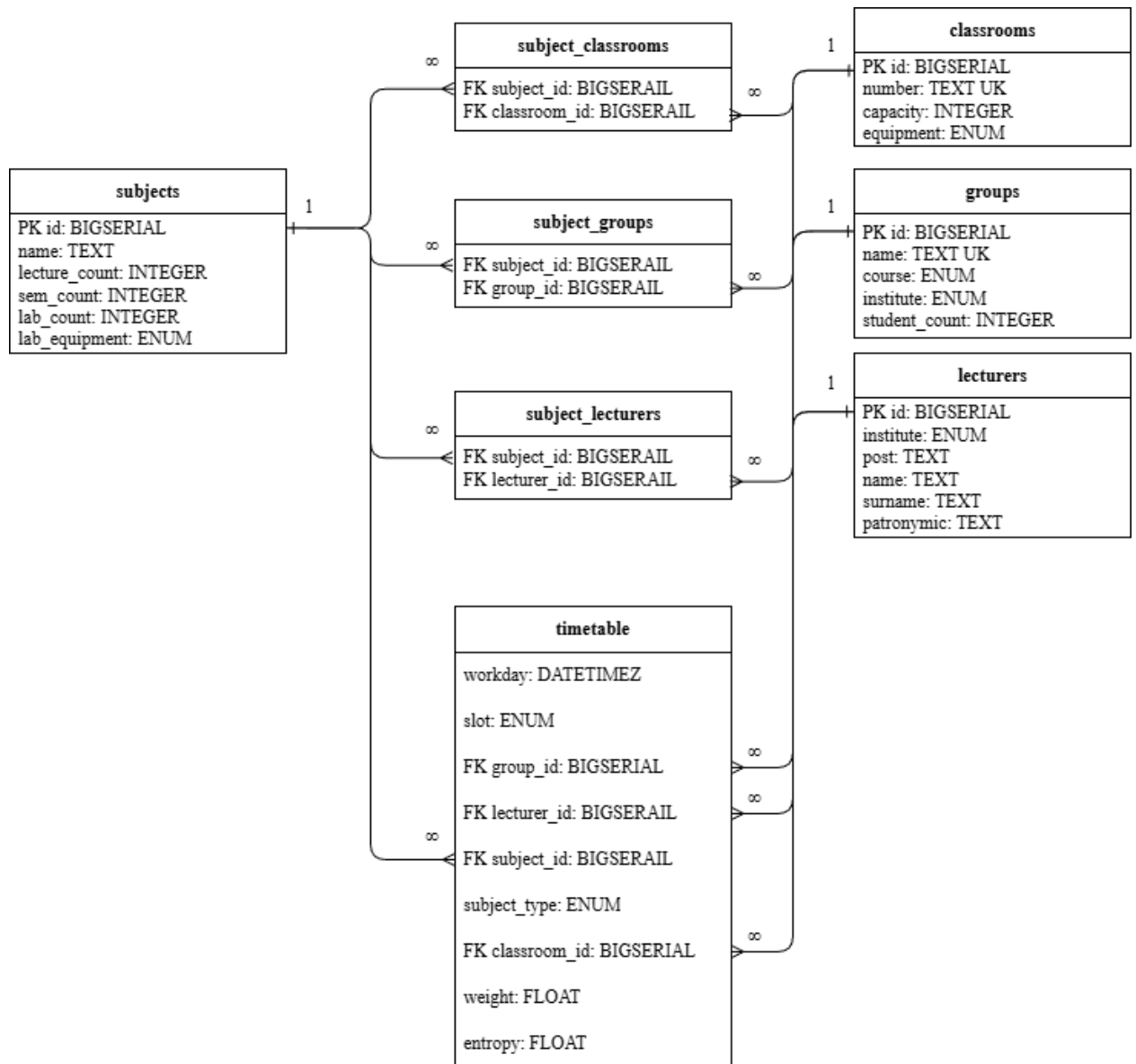


Рис. 3.1. ERD диаграмма базы данных

3.3. ВЫВОД ПО ТРЕТЬЕЙ ГЛАВЕ

Была сформирована архитектура сервиса для генерации расписания учебных занятий. Для системы была выбрана трёхзвенная архитектура, обеспечивающая разделение ответственности между хранилищем данных, серверной логикой и UI. Также была продумана архитектура базы данных и построена ERD диаграмма зависимостей между сущностями.

ГЛАВА 4. ВЫБОР СРЕДСТВ РЕАЛИЗАЦИИ СЕРВИСА ДЛЯ ПРОГРАММНОЙ ПОДДЕРЖКИ ГЕНЕРАЦИИ РАСПИСАНИЯ УЧЕБНЫХ ЗАВЕДЕНИЙ

4.1. ОБОСНОВАНИЕ ВЫБОРА СУБД

При выборе СУБД требуется учесть следующие факторы:

1. СУБД должна быть реляционной. Между таблицами студентов, преподавателей, аудиторий и предметов существует большое количество связей типа «многие ко многим», без обеспечения ссылочной целостности (foreign key integrity) работа с такими данными была бы существенно затруднена.
2. Требуется высокая скорость чтения данных. Как алгоритм WFC, так и генетический алгоритм требуют выполнения большого количества запросов, связанных с выборкой и фильтрацией данных, а также объединение таблиц. При этом характерной особенностью задачи составления учебного расписания является относительно небольшой объём данных: как правило, он составляет считанные мегабайты и полностью помещается в оперативной памяти (RAM).
3. Размер данных: для генерации расписания не требуется огромный объём данных. Учебные планы, списки групп, списки преподавателей и т. д. занимают относительно небольшое место на диске.
4. Тип данных: данные для генерации расписаний являются текстовыми данными и легко могут храниться в любой СУБД или даже просто в текстовом файле, как JSON или CSV.
5. Простота использования: главной задачей ВКР является сам алгоритм генерации расписания, база данных является лишь второстепенной задачей, которая не должна отнимать много времени на развёртывание и заполнение.

Учитывая вышеперечисленные факторы, можно прийти к выводу, что для поставленной задачи требуется небольшая, легковесная БД с высокой

скоростью чтения.

4.1.1. Хранение данных в текстовых файлах

Хранения данных в текстовых файлах, таких как JSON, TXT или CSV отлично подходит для поставленной задачи. Из плюсов можно выделить простоту использования, подходит для хранения сложных, вложенных структур данных, легко читается человеком и легко вносить изменения в структуру данных.

Однако есть и минусы. При каждом чтении будет производиться операция парсинга текстовых данных, что будет делать операции чтения, изменения или добавления сравнительно медленными. Более того, такие, классические для СУБД вещи как поиск, индексация, транзакции отсутствуют, а сами данные будут храниться не в бинарном файле, а в текстовом, что будет неоправданно раздувать объём хранимых данных.

Текстовый формат данных отлично подходит для транспортировки небольших объёмов данных, таких как уже сгенерированное расписание, на клиент для дальнейшего отображения расписания учебных занятий. Однако текстовые файлы не идеально подходят для хранения и работы с данными, используемыми в процессе генерации расписания.

4.1.2. SQLite

SQLite — это самодостаточная и встроенная СУБД [24]. Для развёртывания которой не требуется сервер или сложная система управления. Все данные хранятся в бинарном файле прямо в проекте с расширением “.sqlite” или “.db”.

Благодаря минимальным накладным расходам и высокой скорости выполнения операций чтения SQLite хорошо подходит для задач, в которых данные целиком помещаются в оперативной памяти.

Из минусов часто выделяют ограничение по объёму данных до 140Тб, однако это не будет проблемой для хранения данных для генерации

расписаний учебных занятий. Также SQLite не подходит для сложных многопользовательских сценариев — отсутствует полноценная поддержка конкурентности.

4.1.3. Вывод

При небольшом анализе вариантов СУБД для хранения данных проекта выбор пал на SQLite. Это легковесная СУБД, обладающая преимуществами классических СУБД (индексация, ссылочная целостность, скорость чтения), при этом не требующая отдельного сервера и подходящая для объема данных задачи составления расписания.

4.2. ВЫБОР ВЫЧИСЛИТЕЛЬНОГО ДВИЖКА

Предлагаемый алгоритм решения задачи является ресурсоёмким, поэтому оптимизация производительности системы является одним из ключевых требований. По состоянию на 2026 год одним из наиболее эффективных инструментов для обработки табличных данных в рамках одной машины является библиотека Polars [25].

Изначально Polars был разработан Ричи Винком (Ritchie Vink) как экспериментальный проект во время пандемии COVID-19. Первоначальной целью было создание быстрого парсера CSV-файлов на компилируемом языке Rust, однако со временем проект вырос в полноценный высокопроизводительный движок обработки данных. Polars реализует вычисления с использованием ленивых выражений и оптимизаций на уровне плана выполнения, что позволяет существенно ускорить обработку данных.

Также Polars предоставляет интерфейс для Python, синтаксис которого во многом схож с Pandas. Использование декларативного подхода позволяет абстрагироваться от низкоуровневых деталей реализации таких операций, как агрегация, фильтрация и соединение таблиц (JOIN), что в свою очередь упрощает разработку и повышает читаемость кода.

4.3. ВЫБОР СРЕДСТВ РЕАЛИЗАЦИИ СЕРВЕРА

Поскольку основная вычислительная нагрузка в системе возлагается на движок обработки данных Polars, выбор языка программирования и серверного фреймворка в меньшей степени определяется требованиями к производительности. В данном случае приоритетными становятся такие критерии, как скорость разработки и простота сопровождения.

В качестве языка программирования был выбран Python благодаря его простому синтаксису и широкому набору библиотек. Кроме того, Python имеет нативную интеграцию с Polars, что упрощает организацию обработки данных и снижает накладные расходы на взаимодействие между компонентами системы. Для реализации серверной части был выбран фреймворк FastAPI [26]. Он предоставляет удобные средства для создания REST API, автоматическую валидацию входных данных на основе аннотаций типов, а также встроенную генерацию документации (Swagger).

Дополнительно для управления зависимостями и окружением был использован инструмент UV. Он представляет собой современную альтернативу традиционным решениям, таким как pip и virtualenv, обеспечивая значительно более высокую скорость установки пакетов и воспроизводимость окружения. Использование UV позволяет упростить процесс развёртывания проекта и снизить время настройки среды разработки.

Таким образом, использование Python в сочетании с FastAPI позволяет быстро разрабатывать и масштабировать серверную часть приложения, обеспечивая при этом достаточный уровень производительности и удобство взаимодействия с клиентской частью.

4.4. ВЫБОР СРЕДСТВ РЕАЛИЗАЦИИ КЛИЕНТА

При выборе стека для разработки клиентской части основными критериями являлись современность используемых технологий, удобство

разработки и поддержка типобезопасности. В результате был выбран стек, включающий Nuxt 4, Vue 3.5, TypeScript и пакетный менеджер pnpm.

Фреймворк Nuxt [27] предоставляет высокоуровневую архитектуру для построения приложений на основе Vue. Он включает в себя готовые решения для маршрутизации, серверного рендеринга и управления состоянием, что позволяет сосредоточиться на реализации бизнес-логики, а не на настройке инфраструктуры проекта. Новые версии Nuxt ориентированы на упрощение разработки за счёт автоматической конфигурации и минимизации шаблонного кода.

Использование Vue версии 3.5 обеспечивает доступ к современным возможностям фреймворка, таким как Composition API, улучшенная реактивность и более гибкая организация кода. Это способствует повышению читаемости и переиспользуемости компонентов.

Применение TypeScript добавляет статическую типизацию, что снижает количество ошибок на этапе разработки и упрощает сопровождение проекта. Типизация особенно полезна при взаимодействии с API, где важно строгое соответствие структуры данных.

В качестве пакетного менеджера был выбран pnpm, который отличается высокой скоростью работы и эффективным использованием дискового пространства за счёт механизма кеширования зависимостей. Это ускоряет установку пакетов и делает процесс разработки более предсказуемым. Таким образом, выбранный стек сочетает в себе современные подходы к разработке веб-приложений и удобство использования, позволяя минимизировать технические издержки и сосредоточиться на реализации функциональности UI.

4.5. ВЫВОД ПО ЧЕТВЁРТОЙ ГЛАВЕ

Был сформирован стек технологий, обеспечивающий эффективную реализацию системы генерации расписания учебного заведения. Также был произведён сравнительный анализ подходящих для проекта СУБД и сделан

обоснованный выбор на SQLite.

Выбранные инструменты позволяют решать ключевые задачи проекта: обработку и трансформацию данных, реализацию алгоритмов оптимизации, организацию клиент-серверного взаимодействия и визуализацию результатов. Использование современных решений обеспечивает достаточную производительность системы при работе с большим количеством входных данных.

ГЛАВА 5. РЕАЛИЗАЦИЯ СЕРВИСА ДЛЯ ПРОГРАММНОЙ ПОДДЕРЖКИ ГЕНЕРАЦИИ РАСПИСАНИЯ УЧЕБНЫХ ЗАВЕДЕНИЙ

5.1. ЗАПОЛНЕНИЕ БАЗЫ ДАННЫХ

В идеальном сценарии для заполнения базы данных можно использовать API систем университета, хранящих такие данные как списки групп, преподавателей и учебные планы. Однако на момент написания данной выпускной квалификационной работы такая возможность отсутствует.

Альтернативный вариант — производить парсинг PDF файлов учебных планов и календарных учебных графиков, которые находятся в открытом доступе на сайте университета [20, 21].

Однако существует третий вариант: переиспользовать написанный автором данной выпускной квалификационной работы в бакалавриате парсер [28] PDF файлов расписаний учебных занятий. Хотя это и не будет финальным решением, готовым для запуска системы в работу на пользу учебного заведения — это будет путём наименьшего сопротивления.

Этот подход также решит две другие проблемы, не имеющие элегантного решения. Тогда как списки групп можно получить из соответствующего форума в ЭОС ФГБОУ ВО «МГТУ «СТАНКИН» [29], актуальные списки преподавательского состава или таблицы со всеми учебными аудиториями и оборудованием найти не получится. В случае получения данных сразу из готового расписания за прошлый семестр можно определить и какие предметы ведут какие преподаватели, и какие предметы проводятся в каких аудиториях.

5.1.1. Процесс популяции базы данных

Скачать все расписания на текущий семестр можно в соответствующем форуме в ЭОС ФГБОУ ВО «МГТУ «СТАНКИН» [1]. Там же можно найти и списки групп.

Для заполнения базы данных требуемыми данными была написана специальная вспомогательная программа [30], производящая парсинг данных и приводящая их в нужную структуру и формат в SQLite (см. Рис. 5.1).

	subject_name	lecture_count	sem_count	lab_count	lab_equipment
1	Безопасность жизнедеятельности	12	12	0	"
2	Компьютерная микроскопия	12	6	3	"
3	Компьютерные технологии в приборостроении	14	4	5	"
4	Экономическое обоснование инженерных решений	12	12	0	"
5	Организация и технология испытаний	10	4	3	"
6	Организация и планирование эксперимента	12	6	3	"
7	Системы качества	10	10	0	"
8	Интегрированные системы проектирования и управл...	12	6	3	"
9	Проектирование автоматизированных систем	14	6	4	"
10	Диагностика и надежность автоматизированных сист...	12	12	0	"
11	Организация и планирование автоматизированных п...	8	8	0	"
12	Вычислительные машины, системы и сети	12	4	4	"
13	Автоматные модели в системах управления	8	4	2	"
14	Проектирование роботов и робототехнических систем	12	0	4	"
15	Управление роботами и робототехническими систем...	10	4	3	"
16	Моделирование и исследование робототехнических ...	12	6	3	"
17	САПР робототехнических систем	10	0	3	"
18	Программное обеспечение мехатронных и робототе...	12	4	2	"
19	Компьютерное управление мехатронными системами	10	4	3	"
20	Проектирование мехатронных систем	12	0	4	"
21	Прикладная физическая культура	2	13	0	"
446	Деловые и межкультурные коммуникации	10	10	0	"

Рис. 5.1. Заполненная БД

5.2. СЕРВЕР

Как уже описывалось в предыдущей главе, в качестве основы вычислительной части проекта был выбран Polars. На основе его декларативного языка программирования был реализован алгоритм решения задачи. Все основные методы были обернуты в REST API благодаря FastAPI, а Swagger позволил упростить процесс написания будущего клиента.

Также были разработаны дополнительные инструменты визуализации расписания для упрощения дебага текущего состояния расписания, как например интерактивная визуализация всего расписания на семестр (см. Рис. 5.2).

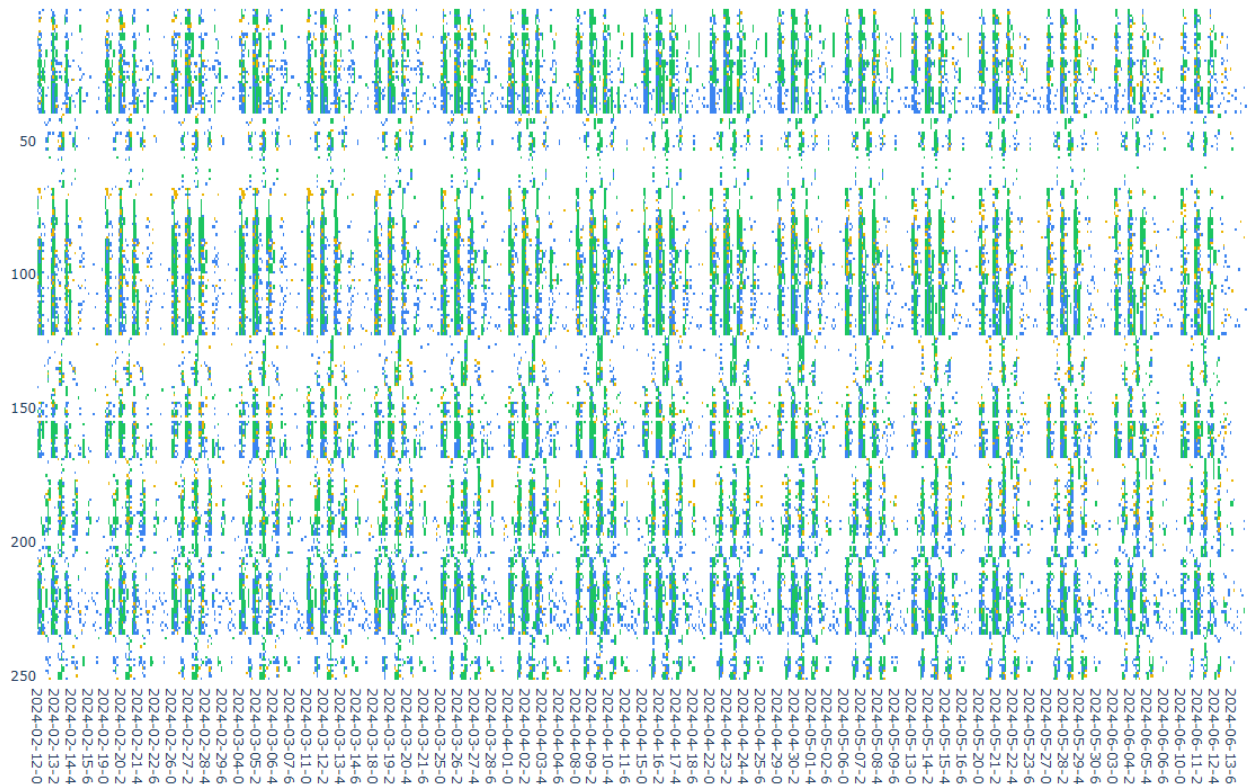


Рис. 5.2. Интерактивная визуализация расписания университета на семестр

5.2.1. Реализация WFC на языке Polars

Первым шагом является генерация первичного расписания, удовлетворяющего минимальным требованиям к расписанию учебных занятий. Этот шаг выполняется методом WFC.

Детальное описание алгоритма приведено ниже.

5.2.1.1. Подготовка таблиц

1. Загрузка из БД множеств групп G , преподавателей L , дисциплин S , а также соответствий «группа–дисциплина» (subject_groups) и «преподаватель–дисциплина» (subject_lecturers).
2. Отфильтровка дисциплин без назначенного преподавателя. После этапа парсинга расписаний встречались предметы без преподавателей, и такие случаи требуется проработать в индивидуальном порядке.
3. Построение множества рабочих дней D : от начала до конца семестра согласно учебному плану, без воскресений.

4. Задание числа пар в день $T = 8$ и нумерация слотов $t \in \{0, \dots, T - 1\}$.
5. Формирование суперпозиции — декартова произведения формула (5.1):

$$\mathcal{C}_G = G \times D \times \{0, \dots, T - 1\}, \quad \mathcal{C}_L = L \times D \times \{0, \dots, T - 1\}. \quad (5.1)$$

где $\mathcal{C}_G$, $\mathcal{C}_L$ — множества ячеек суперпозиции для групп и преподавателей;

G , L — множества групп и преподавателей;

D — множество рабочих дней семестра;

T — число пар в день.

Каждая ячейка (g, d, t) или (ℓ, d, t) изначально свободна ($subject_id = \emptyset$).

6. Вычисление веса ячейки $w(g, d, t)$ по формула (5.2):

$$w(g, d, t) = \log(1 + w_{\text{day}}(\text{weekday}(d))) \cdot w_{\text{slot}}(t), \quad (5.2)$$

где $w(g, d, t)$ — вес ячейки группы;

w_{day} — вес дня недели;

w_{slot} — вес временного слота;

$\text{weekday}(d)$ — день недели для даты d .

С обнулением в праздники; для занятых ячеек $w(g, d, t) = 0$. Именно на этом шаге учитываются предпочтения преподавателей по нагрузке по дням недели, а для студентов движется распределение — дневная или вечерняя форма обучения (см. Рис. 5.3).

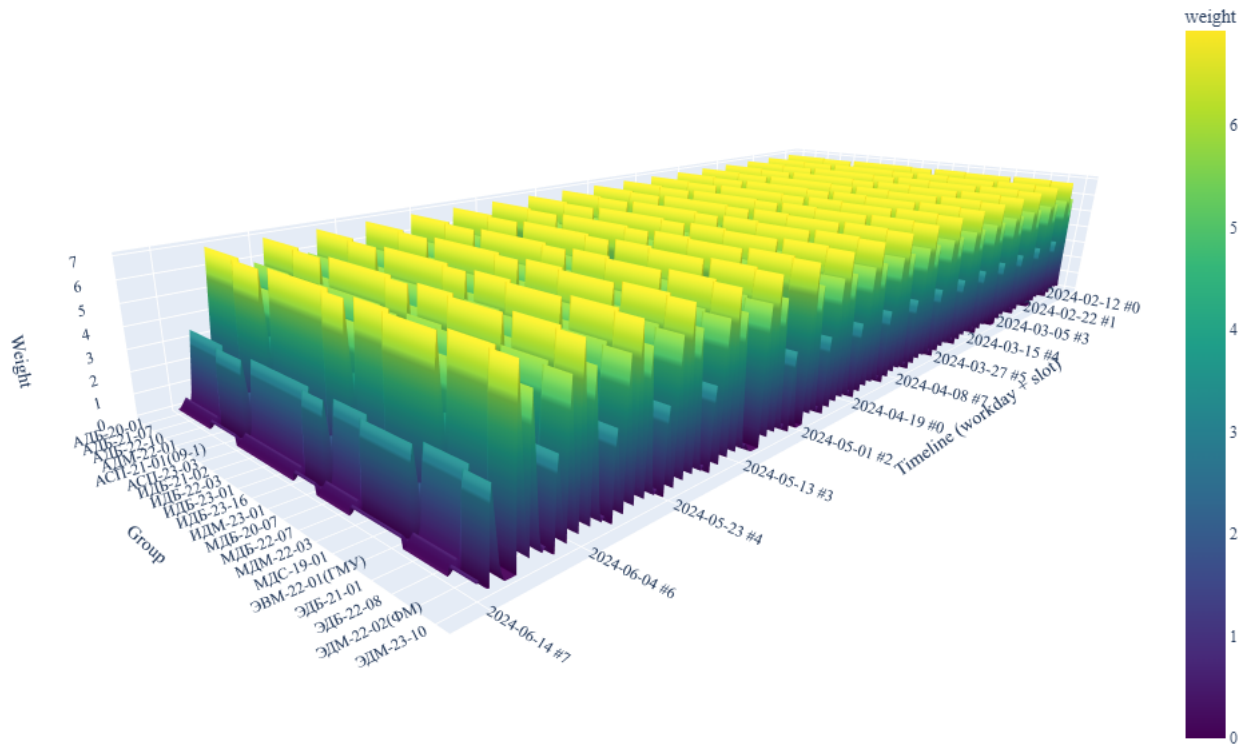


Рис. 5.3. Трёхмерная поверхностная диаграмма распределения весов по временным слотам и группам

7. Вычисление энтропии группы $H(g)$ — суммарного числа оставшихся занятий по всем дисциплинам группы (см. Рис. 5.4) — по формула (5.3):

$$H(g) = \sum_{s \in S(g)} (n_{g,s}^{\text{лек}} + n_{g,s}^{\text{сем}} + n_{g,s}^{\text{лаб}}). \quad (5.3)$$

где $H(g)$ — энтропия группы g ;

$S(g)$ — множество дисциплин группы;

$n_{g,s}^{\text{лек}}, n_{g,s}^{\text{сем}}, n_{g,s}^{\text{лаб}}$ — оставшиеся часы лекций, семинаров и лабораторных по паре (g, s) .

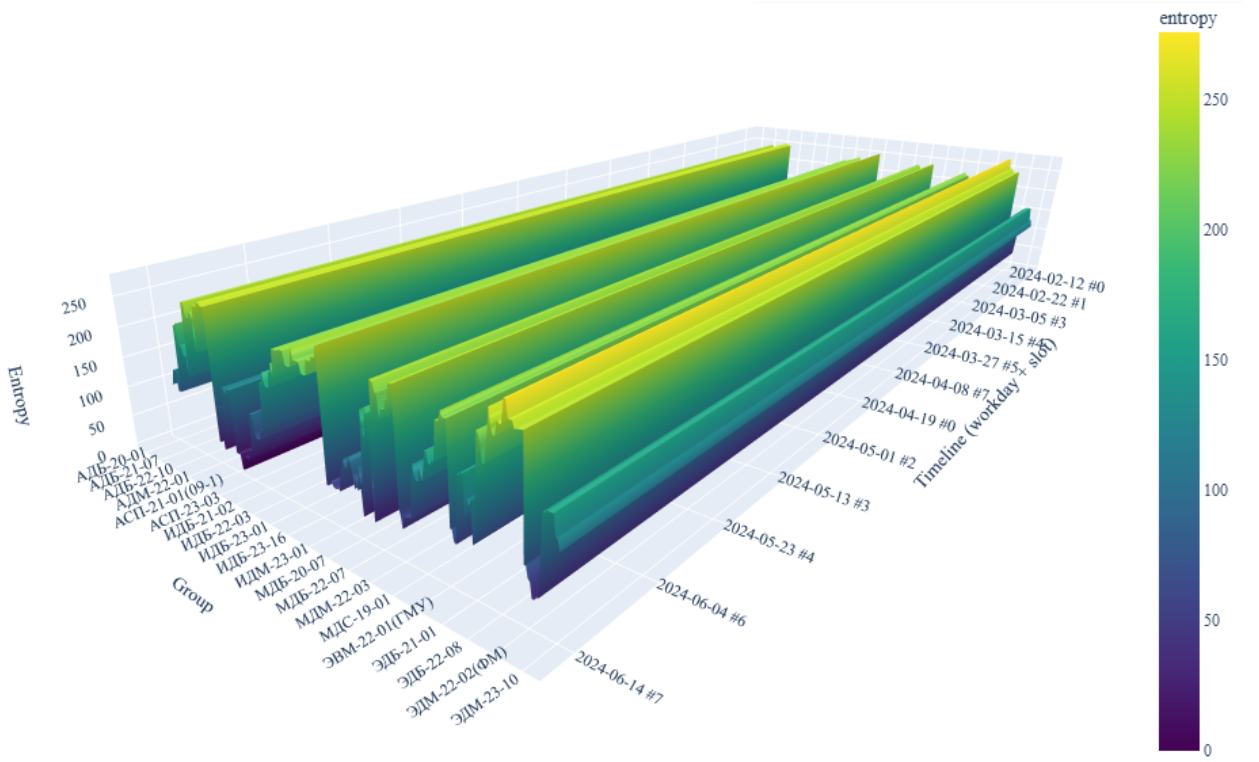


Рис. 5.4. Трёхмерная поверхностная диаграмма распределения энтропии по временным слотам и группам

Аналогично определяется $H(\ell)$ для преподавателей.

5.2.1.2. Итерация WFC (один шаг `apply_one_step`)

Повторять, пока выполняется условие формула (5.4):

$$\sum_{g \in G} \sum_{s \in S(g)} (n_{g,s}^{\text{лек}} + n_{g,s}^{\text{сем}} + n_{g,s}^{\text{лаб}}) > 0. \quad (5.4)$$

где левая часть неравенства — суммарное число нераспределённых занятий по всем группам и дисциплинам.

1. Выбор ячейки для коллапса. Среди свободных ячеек из $\mathcal{C}_G$ с $H(g) > 0$ выбирается множество с минимальной энтропией формула (5.5):

$$\mathcal{M} = \arg \min_{(g,d,t) \in \mathcal{C}_G, H(g) > 0} H(g). \quad (5.5)$$

где $\mathcal{M}$ — множество ячеек с минимальной энтропией $H(g)$.

Затем случайно выбирается $(g^*, d^*, t^*) \in \mathcal{M}$ с вероятностями, пропорциональными $\exp(w - \max w)$ (softmax по весу).

2. Сужение по преподавателям. Сначала формируется множество формула (5.6):

$$L^* = \{\ell \in L : \exists s \in S(g^*), (\ell, s) \in \text{subject_lecturers}\}, \quad (5.6)$$

где L^* — множество допустимых преподавателей;

g^* — выбранная группа;

subject_lecturers — соответствие «преподаватель–дисциплина».

После чего из этого множества исключаются преподаватели, уже занятые в слоте (d^*, t^*) , согласно формула (5.7):

$$L^* \leftarrow \{\ell \in L^* : \text{ячейка } (\ell, d^*, t^*) \text{ свободна}\}. \quad (5.7)$$

где d^*, t^* — выбранные дата и временной слот;

$\leftarrow$ — операция сужения множества L^* .

3. Коллапс дисциплины и типа занятия. Для каждой пары (s, c) , где $c \in \{\text{лек, сем, лаб}\}$, для которой $n_{g^*, s}^c > 0$ и существует $\ell \in L^*$ такой, что $(\ell, s) \in \text{subject_lecturers}$, выбирается одна тройка (s^*, c^*, ℓ^*) взвешенным случайным выбором. Вес кандидата задаётся формула (5.8):

$$p(\ell) \propto \frac{1}{H(\ell) + \varepsilon}, \quad \varepsilon = 10^{-6}. \quad (5.8)$$

где $p(\ell)$ — вес выбора преподавателя ℓ ;

$H(\ell)$ — энтропия преподавателя;

ε — малый положительный параметр, $\varepsilon = 10^{-6}$.

4. Определение целевых групп формула (5.9):

$$G_{\text{tar}} = \begin{cases} \{g^*\}, & c^* \in \{\text{сем, лаб}\}, \\ \left\{ g \in G : \begin{array}{l} \text{группа } g \text{ относится к тому же} \\ \text{потоку, что и } g^* \end{array} \right\}, & c^* = \text{лек}, \end{cases} \quad (5.9)$$

где G_{tar} — целевые группы занятия;

g^*, c^* — выбранные группа и тип занятия;

«поток» — группа групп с общим префиксом имени (например, «ИДМ-24-» для «ИДМ-24-08»), пересечённая с $S(g)$ для дисциплины s^* .

5. Назначение (коллапс). Для всех $g \in G_{\text{tar}}$ в ячейку (g, d^*, t^*) записывается формула (5.10):

$$(s^*, c^*, \ell^*), \quad \text{subject_type} \in \{0, 1, 2\} \Leftrightarrow \{\text{лек, сем, лаб}\}. \quad (5.10)$$

где s^*, c^*, ℓ^* — выбранные дисциплина, тип занятия и преподаватель;
 subject_type — код типа занятия в ячейке расписания.

После этого заносится занятость в ячейку преподавателя (ℓ^*, d^*, t^*) .

6. Локальное распространение ограничений.

- уменьшить $n_{g,s^*}^{c^*}$ на 1 для всех $g \in G_{\text{tar}}$;
- уменьшить $H(g) \leftarrow \max(0, H(g) - 1)$ для всех $g \in G_{\text{tar}}$;
- обнулить w для занятых ячеек.

7. Недельное распространение (flood fill). Пока выполняется условие формула (5.11):

$$\min_{g \in G_{\text{tar}}} n_{g,s^*}^{c^*} > 0, \quad (5.11)$$

где $n_{g,s^*}^{c^*}$ — оставшееся число занятий типа c^* по дисциплине s^* для группы g .

для $k = 1, 2, \dots, 52$ и $\sigma \in \{+1, -1\}$ выполняется попытка поставить то же занятие в $(g, d^* + \sigma k \text{ нед.}, t^*)$ для всех $g \in G_{\text{tar}}$, если:

- дата находится в пределах $[d_{\min}, d_{\max}]$;
- ячейки групп свободны ($\text{subject_id} = \emptyset$);
- преподаватель ℓ^* свободен в этом слоте.

При успешном назначении повторяется п. 6: декрементируются счётчики и обновляется энтропия.

5.2.1.3. Завершение WFC

1. Остановка происходит, когда все счётчики $n^{\text{лек}}, n^{\text{сем}}, n^{\text{лаб}}$ обнуляются.
2. $\mathcal{C}_G$ и $\mathcal{C}_L$ сохраняются как первичное расписание; поле `classroom_id` остаётся пустым ($\emptyset$) и заполняется на отдельном этапе после генетической оптимизации (см. ниже).

5.2.2. Реализация генетического алгоритма на языке Polars

Вторым шагом является улучшение первичного расписания, полученного методом WFC, с помощью эволюционной стратегии. Кроссовер не используется: эволюция строится на отборе, мутациях, сохраняющих допустимость решения, и локальном поиске.

5.2.2.1. Функция приспособленности

Приспособленность задаётся формула (5.12):

$$F(X) = W_{\text{soft}} \cdot S(X) - P_{\text{hard}} \cdot V_{\text{hard}}(X), \quad (5.12)$$

где $F(X)$ — значение функции приспособленности для расписания X ;

W_{soft} — вес мягких критериев, $W_{\text{soft}} = 10^3$;

P_{hard} — штраф за нарушение жёстких ограничений, $P_{\text{hard}} = 10^6$;

$S(X)$ — суммарная оценка мягких критериев;

$V_{\text{hard}}(X)$ — число нарушений жёстких ограничений.

Составляющая $S(X)$ вычисляется по формула (5.13):

$$S(X) = w_{\text{win}} s_{\text{win}}(X) + w_{\text{load}} s_{\text{load}}(X) + w_{\text{start}} s_{\text{start}}(X) + w_{\text{pat}} s_{\text{pat}}(X), \quad (5.13)$$

где $w_{\text{win}}, w_{\text{load}}, w_{\text{start}}, w_{\text{pat}}$ — веса мягких критериев, все равны 1;

$s_{\text{win}}(X)$ — нормированная оценка критерия «окна»;

$s_{\text{load}}(X)$ — нормированная оценка равномерности нагрузки по дням недели;

$s_{\text{start}}(X)$ — нормированная оценка стабильности времени начала занятий;

$s_{\text{pat}}(X)$ — нормированная оценка недельного паттерна.

Жёсткие ограничения (V_{hard}).

Суммарное число нарушений:

- учебный план — расхождение фактического и ожидаемого числа пар по (g, s, c) , а также «лишние» занятия вне плана;
- календарь — занятия вне D или в праздники H_{pr} ;
- слоты — $t \notin \{0, \dots, T - 1\}$;
- конфликты групп — более одного занятия в (g, d, t) ;
- конфликты преподавателей — более одного различного занятия в (ℓ, d, t) ; допускается совместное ведение одной лекции в потоке (одинаковые s и c);
- назначение преподавателя — пара $(\ell, s) \notin \text{subject_lecturers}$ или $\ell = \emptyset$.

Решение допустимо, если $V_{\text{hard}}(X) = 0$.

Мягкие критерии (S).

Каждый критерий нормируется в $(0, 1]$ по формула (5.14):

$$s = \frac{1}{1 + \text{метрика}}, \quad (5.14)$$

где s — нормированная оценка критерия;

метрика — исходное значение соответствующего критерия качества.

1. Окна (s_{win}). Для каждой пары (g, d) считается число «пустых» слотов

между первой и последней парой дня по формула (5.15):

$$W_{\text{tot}} = \sum_{g,d} \left(\max_{t \in X(g,d)} t - \min_{t \in X(g,d)} t + 1 - |X(g,d)| \right). \quad (5.15)$$

где W_{tot} — суммарное число «окон» в расписании;

$X(g, d)$ — множество занятых слотов группы g в день d .

2. Равномерность нагрузки по дням недели (s_{load}). Для группы g вычисляется коэффициент вариации числа пар по дням недели; затем усредняется по группам.
3. Стабильность времени начала (s_{start}). Для каждой тройки (g, s, c) берётся стандартное отклонение слота t по всем неделям; затем усредняется.
4. Недельный паттерн (s_{pat}). Для $(g, s, c, \text{weekday}(d))$ штрафуются использование более одного различного слота по формула (5.16):

$$P_{\text{pat}} = \sum \max(|\{t\}| - 1, 0). \quad (5.16)$$

где P_{pat} — штраф за нарушение недельного паттерна;

$\{t\}$ — множество слотов, используемых для данной комбинации $(g, s, c, \text{weekday}(d))$.

5.2.2.2. Операторы мутации

Все операторы перемещают одно занятие из ячейки (g, d, t) в (g, d', t') при выполнении предиката `can_place`:

- $d' \in D \setminus H_{\text{pr}}$;
- целевая ячейка группы свободна;
- преподаватель ℓ свободен в (d', t') либо ведёт то же (s, c) (потокковая лекция);
- $(\ell, s) \in \text{subject_lecturers}$.

Вариации алгоритма SA:

1. `holiday_repair` — перенос занятия с праздничной даты на случайный рабочий день и слот.
2. `relocate_session` — случайный перенос произвольного назначенного занятия.
3. `compress_day` — выбор дня (g, d) с окнами $(\max t - \min t + 1 - |\{t\}| > 0)$, выбор пустого слота внутри дневного интервала и сдвиг крайнего занятия в этот слот.
4. `align_pattern_slot` — для «размазанного» паттерна $(g, s, c, \text{weekday})$ выбирается целевой слот (медиана по текущим слотам) и одно занятие переносится на него в одном из соответствующих рабочих дней.

Случайный оператор выбирается равновероятно; при наличии праздничных занятий с вероятностью 0,5 приоритетно вызывается `holiday_repair`. Каждая попытка — до 40 случайных кандидатов (d', t') .

5.2.2.3. Инициализация популяции

Размер популяции N_{pop} .

1. Элита: seed-расписание WFC без изменений.
2. Остальные $N_{\text{pop}} - 1$ особи: копии seed с 120 случайными мутациями (с приоритетом `holiday_repair`).
3. Каждая особь оценивается функцией F ; популяция сортируется по убыванию приспособленности.

5.2.2.4. Итерация эволюции

Повторять до G_{max} поколений или до плато без улучшения лучшего решения.

1. Элитизм: лучшие особи переносятся в следующее поколение без изменений.
2. Отбор родителя: турнирный отбор ($k = 3$) — из k случайных особей выбирается лучшая по F .

3. Потомок: клон родителя; к нему применяется 2 случайные мутации.
4. Локальный поиск (hill climbing): до 30 попыток случайной мутации; принимается только улучшение или сохранение F . Это ускоряет сходимость без нарушения допустимости.
5. Формирование нового поколения: все потомки оцениваются, популяция снова сортируется по F .
6. Глобальный лучший: если F лучшей особи поколения превышает текущий рекорд, обновляется $best$; иначе счётчик плато увеличивается на 1.

5.2.2.5. Завершение генетического алгоритма

1. Остановка по достижению плато или лимита поколений.
2. Сохранение лучшего расписания (см. Рис. 5.5).

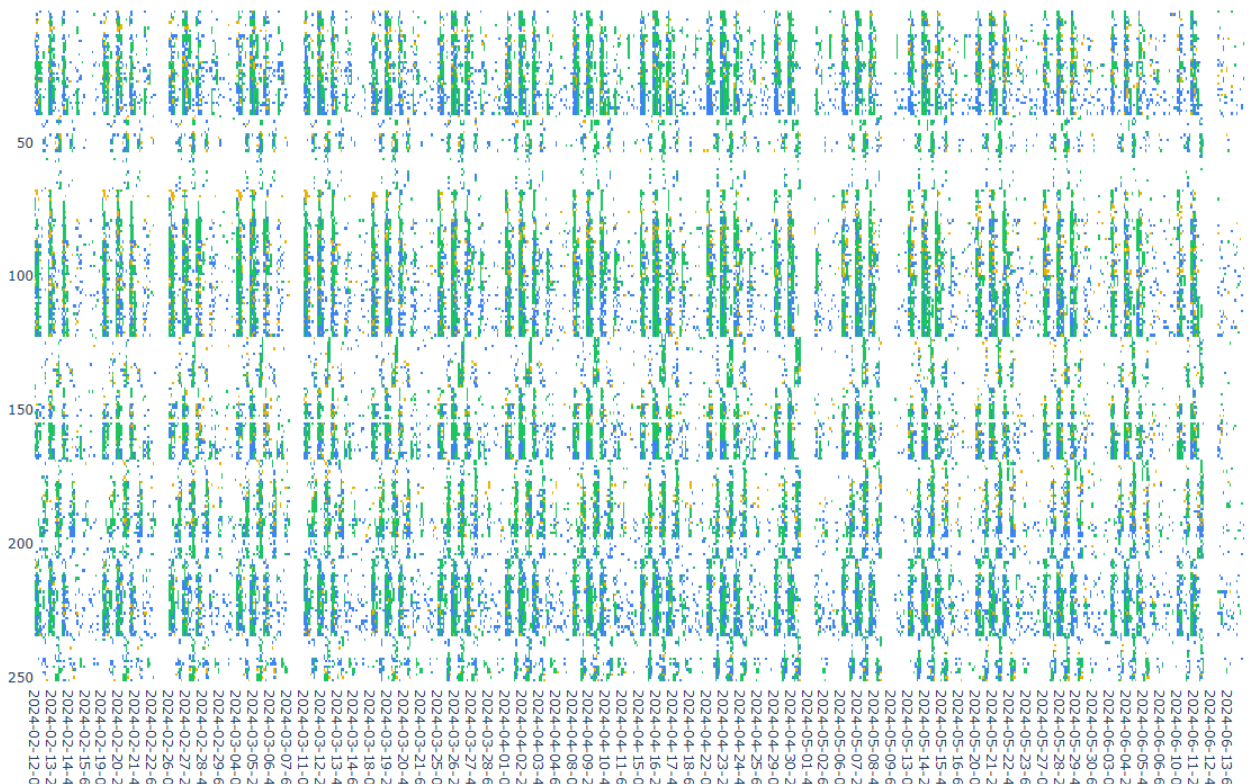


Рис. 5.5. Результат генетического алгоритма после 1000 поколений

5.3. ЭКСПЕРИМЕНТАЛЬНАЯ ОЦЕНКА АЛГОРИТМА

Для проверки работоспособности предложенного конвейера выполнены вычислительные эксперименты на данных МГТУ «СТАНКИН», полученных из расписания прошлого семестра (см. подраздел о наполнении базы данных).

5.3.0.1. Время выполнения

Построение первичного расписания методом WFC на использованной конфигурации сервера занимает порядка 6 часов. Эволюционная оптимизация в режиме 1000 поколений выполняется около 24 часов; по графику изменения целевой функции (см. Рис. 5.5) значение приспособленности к концу прогона продолжает монотонно улучшаться, то есть алгоритм не достигает плато в пределах заданного лимита поколений.

5.3.0.2. Сравнение с ручным расписанием

Для сопоставления качества решений все варианты оценены единой целевой функцией F (формулы формула (5.12)–формула (5.13)), включая расписание, составленное вручную сотрудниками УМУ и загруженное в систему (см. Рис. 5.6). Помимо итогового значения F , зафиксированы диагностические показатели, напрямую влияющие на мягкие критерии (см. Таблица 5.1).

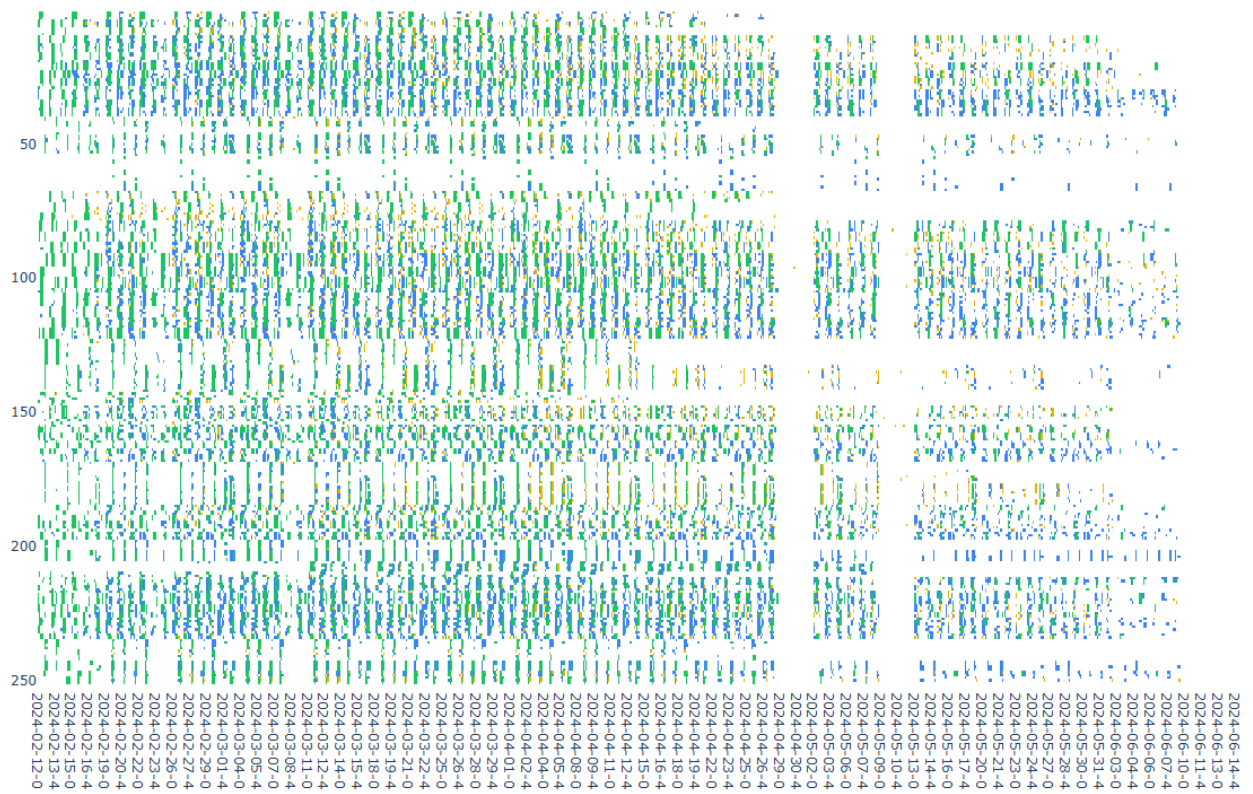


Рис. 5.6. Насписание, составленное сотрудниками УМУ

Таблица 5.1.
Сравнение расписаний по диагностическим метрикам

Вариант расписания	Занятий в праздники	Число окон	Рабочих дней в неделю
Оригинальное (ручное)	459	3 994	6
После WFC	1 306	5 815	5
После генетического алгоритма (1000 поколений)	0	6 155	5

Ручное расписание содержит меньше окон, однако допускает занятия в праздничные дни и использует шестидневную учебную неделю. Расписание после WFC удовлетворяет жёстким ограничениям по учебному плану, но мягкие критерии остаются неудовлетворительными: значительное число занятий в праздники и повышенное число окон. Эволюционная оптимизация устраняет все зафиксированные занятия в праздники (0), при этом число окон возрастает относительно ручного варианта — целевая функция на данном

этапе в большей степени штрафует праздничные и календарные нарушения, чем окна, а лимит поколений не исчерпан по критерию плато.

Таким образом, гибридный конвейер WFC + эволюционная оптимизация демонстрирует практическую применимость: конструктивный этап формирует допустимую основу, а последующая оптимизация систематически исправляет нарушения, критичные для регламента вуза. Дальнейшее снижение числа окон требует увеличения числа поколений или перенастройки весов w_* в функции $S(X)$.

5.3.1. Назначение аудиторий

Третьим шагом к уже оптимизированному расписанию групп (после WFC и генетического алгоритма) последовательно назначаются аудитории. Этот этап не участвует в эволюции: временная структура расписания фиксирована, меняется только поле `classroom_id`.

5.3.1.1. Подготовка

1. Загрузка множества аудиторий A с атрибутами $\text{cap}(a)$ (вместимость) и $\text{equip}(a)$ (оборудование).
2. Предварительное вычисление матрицы кратчайших расстояний $\text{Dist}(a_i, a_j)$ между аудиториями (см. Рис. 5.7).

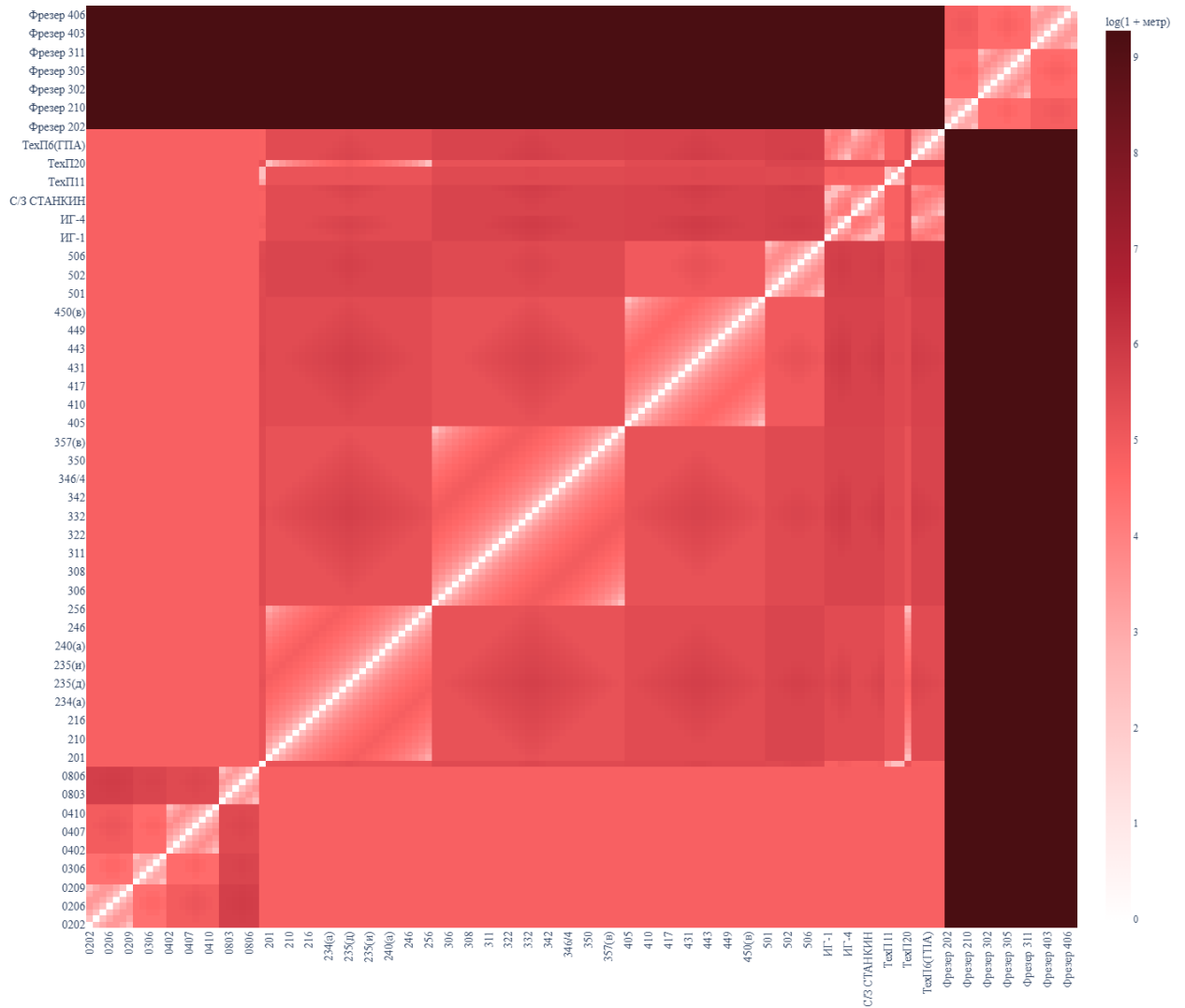


Рис. 5.7. Матрица примерных расстояний между аудиториями

5.3.1.2. Назначение по занятиям

Для каждого назначенного занятия (g, d, t, s, c, ℓ) в порядке обхода расписания строится множество допустимых аудиторий формула (5.17):

$$A_{\text{adm}}(g, d, t, s, c) = \{a \in A : \text{cap}(a) \geq |G_{\text{tar}}|, \text{equip}(a) \supseteq \text{lab_equipment}(s, c), a \text{ свободна}\} \quad (5.17)$$

где $A_{\text{adm}}(g, d, t, s, c)$ — множество допустимых аудиторий для занятия;

g — учебная группа;

d — дата занятия;

t — номер временного слота;

s — дисциплина;

c — тип занятия (лекция, семинар, лабораторная);

a — аудитория из множества A ;

$\text{cap}(a)$ — вместимость аудитории;

$\text{equip}(a)$ — оборудование аудитории;

G_{tar} — целевые группы занятия (одна группа для семинара или лабораторной, поток — для лекции).

Выбор аудитории выполняется жадным образом с учётом уже назначенных занятий группы g в тот же день d :

- если для дня d это первое занятие группы g , выбирается первая аудитория из A_{adm} , отсортированного в лексикографическом порядке по номеру аудитории;
- если в этот день уже назначена аудитория a_{prev} , множество A_{adm} сортируется по возрастанию $\text{Dist}(a_{\text{prev}}, a)$, и выбирается первая допустимая аудитория.

Так достигается двойная цель: минимизация числа используемых аудиторий за счёт закрепления кабинетов за группами и минимизация перемещений внутри корпуса за счёт локальной оптимизации переходов между последовательными парами.

5.4. КЛИЕНТ

В роли клиента реализовано веб-приложение на базе Nuxt 4 и Vue 3.5 (см. Рис. 5.8). Технология PWA позволяет установить сайт как приложение на мобильные устройства и использовать сервис без установки отдельного нативного клиента.



Рис. 5.8. Клиент

5.5. ВЫВОД ПО ПЯТОЙ ГЛАВЕ

Была реализована программная основа сервиса для генерации расписания учебных занятий. Для системы реализована трёхзвенная архитектура, обеспечивающая разделение ответственности между хранилищем данных, серверной логикой и UI. Подготовлен механизм заполнения БД на основе доступных источников данных.

На стороне сервера реализованы алгоритмы WFC и эволюционной оптимизации, REST API и средства отладочной визуализации. Эксперименты показали: WFC формирует допустимое расписание за 6 часов; за 1000 поколений (24 часа) генетический алгоритм устраняет занятия в праздники относительно результата WFC, при этом целевая функция продолжает улучшаться. На стороне клиента разработано веб-приложение с поддержкой PWA.

В результате получена работоспособная система, пригодная для

дальнейшего развития и интеграции в процесс автоматизированного формирования расписания вуза.

ГЛАВА 6. ЭКОНОМИЧЕСКОЕ ОБОСНОВАНИЕ

Актуальность автоматизации процесса составления расписания подтверждается не только внутренними затратами учебного заведения, но и запросом студенческого сообщества. Во «ВКонтакте» размещена петиция «Расписание за неделю» [31], в которой обучающиеся МГТУ «СТАНКИН» фиксируют потребность в своевременной и полной публикации расписания занятий на неделю вперёд, а не фрагментарном или запоздалом доведении информации до студентов и преподавателей. Подобные обращения свидетельствуют о том, что качество и оперативность расписания напрямую влияют на удовлетворённость участников образовательного процесса и воспринимаются как значимый фактор организации учебной деятельности вуза.

С экономической точки зрения такой запрос означает необходимость сократить цикл подготовки и пересборки расписания, снизить число ошибок при ручном согласовании и обеспечить единый канал публикации результата. Разрабатываемое программное средство направлено на устранение этих проблем и тем самым отвечает как организационным потребностям УМУ, так и ожиданиям контингента, выраженным в публичной инициативе.

6.1. АНАЛИЗ АНАЛОГОВ И ОБОСНОВАНИЕ СОБСТВЕННОГО РЕШЕНИЯ

Для оценки целесообразности разработки и внедрения программного средства поддержки генерации расписания учебных занятий необходимо рассмотреть существующие подходы к решению данной задачи в образовательных организациях.

В МГТУ «СТАНКИН» на момент выполнения выпускной квалификационной работы расписание на семестр формируется вручную сотрудниками УМУ [1]. Данный процесс является базовым аналогом: он не требует лицензий на программное обеспечение, но связан с

высокими трудозатратами, риском ошибок при согласовании ограничений и длительным циклом пересборки расписания при изменении исходных данных.

На российском рынке представлены специализированные системы автоматизированного составления расписания для вузов.

Галактика РУЗ [32] — комплексное решение для управления учебным процессом в высших учебных заведениях. Система поддерживает автоматическое формирование опорного расписания, контроль параметризуемых ограничений, публикацию результатов на сайте вуза и в мобильных приложениях. Внедрение предполагает закупку лицензий и сопутствующей инфраструктуры.

1С:Автоматизированное составление расписания. Университет [33] — продукт экосистемы 1С для автоматизированного, ручного и смешанного составления расписаний с учётом нагрузки преподавателей, аудиторного фонда и учебных планов. Типовое внедрение включает лицензирование, настройку под регламенты вуза и интеграцию с другими модулями 1С.

ТАНДЕМ.Университет (модуль «Расписание») [34] — модуль корпоративной системы управления вузом. Обеспечивает автоматическое и ручное составление расписания, фиксацию ограничений преподавателей и требований к аудиториям, поддержку индивидуальных образовательных траекторий.

Экспресс-расписание ВУЗ [35] — программа для автоматического составления расписания в локальной или сетевой конфигурации. Поддерживает работу с подгруппами, планирование отсутствия преподавателей и публикацию расписания на сайте образовательной организации.

Зарубежные системы UniTime и TimeEdit, широко применяемые в зарубежных вузах, рассматриваются в обзорной литературе по задаче составления расписаний [4], однако их внедрение в российский вуз сопряжено с адаптацией нормативной базы, локализацией и интеграцией с

отечественными информационными системами.

Разрабатываемое в рамках данной выпускной квалификационной работы программное средство не претендует на полную замену корпоративных ERP-систем, но ориентировано на узкую задачу — генерацию и оптимизацию расписания с учётом правил МГТУ «СТАНКИН», предпочтений преподавателей, «компоновки вуза» и публикации результата через веб-интерфейс и PWA-клиент. Использование открытого программного стека (Python, FastAPI, Polars, SQLite, Nuxt) снижает стоимость владения по сравнению с коммерческими аналогами.

На основе описания функциональности перечисленных решений проведён сравнительный анализ (см. Таблица 6.1).

Таблица 6.1.

Сравнительная таблица функций аналогов и целевого программного продукта

Критерий	Ручной процесс	Коммерч. системы	Продукт ВКР
Автогенерация с учётом нормативов	—	+	+
Учёт предпочтений преподавателей	±	+	+
Оптимизация перемещений (компоновка)	—	±	+
Адаптация под правила вуза	+	±	+
Низкая стоимость владения	+	—	+
Веб-доступ и PWA	±	+	+

Целевой программный продукт превосходит ручной процесс по скорости пересборки расписания и снижению числа конфликтов ограничений. По сравнению с коммерческими ERP-модулями он обеспечивает специализацию под регламенты МГТУ «СТАНКИН» и контролируемую совокупную стоимость владения за счёт открытого программного обеспечения. Таким образом, разработка собственного решения экономически оправдана как альтернатива как полностью ручному составлению, так и закупке дорогостоящего типового продукта без гарантии

покрытия всех специфических требований вуза.

6.2. ОЦЕНКА ЗАТРАТ ПРОЕКТА

Для оценки затрат на создание и внедрение программного средства определены следующие ключевые ресурсы:

- разработчик (full-stack, автор ВКР) — проектирование, реализация серверной и клиентской частей, алгоритмов генерации;
- специалист по внедрению и сопровождению (системный администратор) — развёртывание в контуре вуза, обучение пользователей, техническая поддержка;
- программное обеспечение — операционная система, среда разработки, серверные компоненты;
- инфраструктура и оборудование — сервер приложений и рабочее место администратора.

Затраты на разработку оцениваются ретроспективно по этапам, отражённым в главах 3–5 выпускной квалификационной работы, с учётом доработок после защиты. Затраты на внедрение и эксплуатацию относятся к первому году использования системы в МГТУ «СТАНКИН».

Для расчёта приняты следующие ставки оплаты труда: разработчик — 1000 руб./ч, системный администратор — 500 руб./ч

Расчёт прямых трудозатрат приведён в Таблица 6.2.

Таблица 6.2.
Расчёт прямых трудовых затрат

Наименование работ	Часы	Специалист	Затраты, руб.
Проектирование архитектуры и БД (гл. 3)	80	Разработчик	80 000
Выбор и настройка стека (гл. 4)	40	Разработчик	40 000
Парсер и наполнение БД (гл. 5)	60	Разработчик	60 000
Сервер: WFC, оптимизация, API (гл. 5)	140	Разработчик	140 000
Клиент Nuxt, PWA (гл. 5)	80	Разработчик	80 000
Тестирование и отладка (гл. 5)	60	Разработчик	60 000
Доработки после защиты ВКР	40	Разработчик	40 000
Итого разработка	500		500 000
Развёртывание в контуре вуза	60	Администратор	30 000
Обучение сотрудников УМУ	40	Администратор	20 000
Поддержка (10 ч/мес × 12)	120	Администратор	60 000
Итого внедрение и эксплуатация	220		110 000

Общая сумма расходов на трудовые затраты составляет 610 000 руб., общая трудоёмкость — 720 чел.-часов.

Основная часть программного обеспечения проекта распространяется бесплатно (см. Таблица 6.3), что соответствует выбранному в главе 4 стеку технологий.

Таблица 6.3.
Расчёт затрат на программное обеспечение

Наименование	Стоимость, руб.	Примечание
Python, FastAPI, Polars, SQLite	0	Открытый исходный код
Nuxt, Vue, TypeScript, pnpm	0	Открытый исходный код
Ubuntu Server, Git, VS Code	0	Бесплатное ПО
Лицензия на антивирус	5 000	Стоимость на 1 год

Для размещения серверной части и работы администратора необходимо серверное и клиентское оборудование (см. Таблица 6.4). В отличие от типовых смет внедрения учётных систем, в проекте не закладывается лицензия Windows Server — используется Linux.

Таблица 6.4.

Расчёт затрат на инфраструктуру

Часть	Наименование	Кол-во, ед.	Стоимость, руб.
Серверная	Сервер приложений (8 ядер, 32 ГБ ОЗУ, SSD)	1	100 000
Клиентская	ПК администратора (16 ГБ ОЗУ)	1	65 000
Серверная	Сетевое оборудование	1	2 000
Итого			167 000

Амортизация оборудования рассчитывается линейным методом по формула (6.1). Срок полезного использования принят равным 7 годам:

$$A_{\text{год}} = \frac{C_{\text{обор}}}{T_{\text{сл}}}, \quad (6.1)$$

где $A_{\text{год}}$ — годовая сумма амортизации, руб.;

$C_{\text{обор}}$ — первоначальная стоимость оборудования, руб.;

$T_{\text{сл}}$ — срок полезного использования, лет.

Годовая сумма амортизации составляет $167\,000/7 \approx 24\,000$ руб. Для периода внедрения в первый год (5 месяцев) в смету включена доля амортизации 15 000 руб. (см. Таблица 6.5).

Таблица 6.5.

Расчёт затрат на амортизацию (годовые показатели)

Наименование	Стоимость, руб.	Амортизация в год, руб.
Сервер приложений	100 000	14 286
ПК администратора	65 000	9 286
Сетевое оборудование	2 000	286
Итого	167 000	24 000

Затраты на электроэнергию за год оцениваются по формула (6.2):

$$Z_{\text{эл}} = P \cdot t \cdot c, \quad (6.2)$$

где $Z_{эл}$ — затраты на электроэнергию за год, руб.;

P — потребляемая мощность, кВт;

t — время работы оборудования в год, ч;

c — тариф, руб./(кВт·ч).

Для сервера и рабочего места администратора суммарная мощность принята равной 0,5 кВт при 1600 ч работы в год и тарифе 5,47 руб./(кВт·ч):
 $Z_{эл} = 0,5 \cdot 1600 \cdot 5,47 \approx 4\,400$ руб./год.

Затраты на ремонт и обслуживание оборудования оцениваются в 10 % от его стоимости в год: $167\,000 \cdot 0,1 = 16\,700$ руб.

Исходя из проведённых вычислений, сформирована общая смета затрат на первый год внедрения (см. Таблица 6.6).

Таблица 6.6.
Общая смета затрат на проект (первый год)

Статья затрат	Сумма, руб.
Трудозатраты разработки (ВКР и доработки)	500 000
Трудозатраты внедрения и поддержки	110 000
Программное обеспечение	5 000
Инфраструктура	167 000
Амортизация (доля за период внедрения)	15 000
Электроснабжение и ремонт	21 100
Итого	818 100

6.3. ОБОСНОВАНИЕ ЭКОНОМИЧЕСКОЙ ВЫГОДЫ ПРОЕКТА

Внедрение программного средства поддержки генерации расписания обеспечит следующие экономические преимущества для МГТУ «СТАНКИН»:

- сокращение ручного труда сотрудников УМУ при пересборке расписания на семестр;
- снижение числа конфликтов по аудиториям, нагрузке преподавателей и «окнам» в расписании;

- ускорение публикации актуального расписания через веб-интерфейс и PWA вместо разрозненных файлов;
- учёт предпочтений преподавателей и оптимизация перемещений между корпусами без многократных ручных согласований.

Для количественной оценки эффекта используются допущения. Базовые трудозатраты УМУ на ручное составление расписания оцениваются так: $3 \text{ сотрудника} \times 120 \text{ ч} \times 500 \text{ руб./ч} = 180\,000 \text{ руб.}$ за семестр, или $360\,000 \text{ руб.}$ в год (два семестра).

Автоматизация позволяет сократить трудозатраты на подготовку и корректировку расписания на 35–40 %. При коэффициенте сокращения $k = 0,375$ годовая экономия определяется формула (6.3):

$$E_{\text{год}} = Z_{\text{баз}} \cdot k = 360\,000 \cdot 0,375 = 135\,000 \text{ руб.} \quad (6.3)$$

где $E_{\text{год}}$ — годовая экономия трудозатрат, руб.;

$Z_{\text{баз}}$ — базовые годовые трудозатраты УМУ без автоматизации, руб.;

k — коэффициент сокращения трудозатрат.

Сравнение с закупкой коммерческого решения для вуза сопоставимого масштаба (по аналогии с оценками внедрения типовых учётных систем) показывает, что совокупные затраты на лицензию, кастомизацию и внедрение могут составить 1,5–2,5 млн руб. Разработка и внедрение собственного средства по смете Таблица 6.6 обходятся примерно в 818 100 руб., то есть прямая экономия на капитальных затратах (CAPEX) по сравнению с коммерческим аналогом составляет порядка 0,7–1,7 млн руб. при сопоставимом базовом функционале генерации расписания.

Срок окупаемости проекта по операционной экономии УМУ определяется формула (6.4):

$$T_{\text{ок}} = \frac{Z_{\text{проект}}}{E_{\text{год}}} = \frac{818\,100}{135\,000} \approx 6,1 \text{ года.} \quad (6.4)$$

где $T_{\text{ок}}$ — срок окупаемости проекта, лет;

$Z_{\text{проект}}$ — затраты на проект по смете, руб.;

$E_{\text{год}}$ — годовая экономия трудозатрат, руб..

При учёте отказа от закупки коммерческой лицензии совокупный экономический эффект превышает затраты на проект уже в первый год эксплуатации.

Дополнительно повышается качество расписания: уменьшение «окон», равномерность нагрузки и учёт «компоновки вуза» снижают косвенные потери времени студентов и преподавателей, не полностью отражённые в расчёте $E_{\text{год}}$.

6.4. ВЫВОД ПО ЭКОНОМИЧЕСКОМУ ОБОСНОВАНИЮ

Разработка и внедрение программного средства поддержки генерации расписания для МГТУ «СТАНКИН» экономически целесообразны как альтернатива полностью ручному процессу и закупке дорогостоящих типовых ERP-модулей. Низкая совокупная стоимость владения обеспечивается использованием открытого программного обеспечения; операционная экономия формируется за счёт сокращения трудозатрат УМУ. Ограничением остаётся необходимость доработки интеграции с корпоративными системами вуза (ЭОС, учебные планы), отражённая в главе 5. Для небольших подразделений с минимальным объёмом расписания внедрение может быть менее выгодным; для вуза уровня МГТУ «СТАНКИН» проект является обоснованным с точки зрения соотношения затрат и эффекта.

ЗАКЛЮЧЕНИЕ

1. Выпускная квалификационная работа посвящена повышению эффективности деятельности учебного заведения за счёт автоматизации подготовки расписания учебных занятий. Для МГТУ «СТАНКИН» актуальны сокращение трудозатрат УМУ при пересборке расписания, снижение риска ошибок при согласовании ограничений и обеспечение удобной публикации результата для преподавателей и студентов. Целью работы является исследование методов автоматической генерации расписания и разработка программного средства, реализующего такую генерацию с учётом регламента вуза, предпочтений преподавателей и «компоновки вуза», а также веб-доступа к результату.
2. В первой главе выполнен анализ предметной области и существующих подходов к автоматическому составлению расписаний. Показано, что задача относится к классу NP-hard; рассмотрены эвристики, метаэвристики (одиночные и популяционные) и гиперэвристики. Обоснован гибридный метаэвристический конвейер: WFC для допустимого расписания, эволюционная оптимизация с локальным поиском для мягких критериев, жадное назначение аудиторий.
3. Во второй главе систематизированы минимальные и желательные требования к расписанию, критерии его оценки и ограничения, связанные с учебными планами, календарём и нормативами. Предложен метод генерации: первичное расписание, удовлетворяющее жёстким ограничениям, формируется алгоритмом коллапса волновой функции (WFC); дальнейшее улучшение выполняется эволюционной схемой на основе генетического алгоритма (отбор, мутации, сохраняющие допустимость, локальный поиск и штрафование нарушений жёстких ограничений). Таким образом, реализован настраиваемый баланс между соблюдением регламента и мягкими целями (окна, равномерность, паттерны и др.).

4. В третьей и четвёртой главах обоснована архитектура программного средства (хранилище данных, серверная логика, клиент) и выбран стек технологий: SQLite как СУБД, Polars для ресурсоёмкой обработки данных, FastAPI и REST API на сервере, клиент на базе Nuxt и Vue с поддержкой PWA. Выбор согласован с требованиями к производительности, сопровождаемости и стоимости владения.
5. В пятой главе описана реализация сервиса: наполнение базы данных из доступных источников, алгоритм WFC на декларативных операциях Polars, эволюционная оптимизация расписания и последующее назначение аудиторий, серверное API и клиентское веб-приложение. Экспериментально подтверждена работоспособность конвейера ($WFC \approx 6$ ч, 1000 поколений эволюции ≈ 24 ч); сравнение с ручным расписанием по единой целевой функции показало устранение занятий в праздники после эволюционного этапа. Получено работоспособное программное средство, пригодное для дальнейшего развития и интеграции в процесс подготовки расписания вуза.
6. В главе с экономическим обоснованием сопоставлены затраты на разработку и внедрение собственного решения с ручным процессом и коммерческими аналогами; показана целесообразность проекта для вуза масштаба МГТУ «СТАНКИН» при использовании открытого программного обеспечения и оценены ориентиры по операционной экономии и сроку окупаемости.
7. Поставленные в задании на ВКР задачи выполнены: проведён анализ методов и обоснован собственный подход, сформулированы требования к клиент-серверному приложению, разработаны проект и реализация веб-сервиса генерации расписания. Направления дальнейшей работы включают углублённую интеграцию с корпоративными системами вуза (в том числе ЭОС и учебные планы), расширение источников входных данных и доведение системы до режима промышленной эксплуатации с учётом регламентов информационной безопасности.

СПИСОК ЛИТЕРАТУРЫ

1. Расписание занятий [Электронный ресурс] / ФГБОУ ВО «МГТУ «СТАНКИН»». — URL: <https://edu.stankin.ru/course/view.php?id=11557> (дата обр. 18.12.2024).
2. Кочановский, К. Материалы студенческой научно-практической конференции «Автоматизация и информационные технологии (АИТ-2024)» / К. Кочановский // Сборник I тура ИИТ. — 2024. — С. 139. — URL: https://old.stankin.ru/uploads/files/file_6657395e57b82.pdf (дата обр. 20.05.2026).
3. Кочановский, К. Алгоритм решения задач составления расписания занятий в образовательных учреждениях / К. Кочановский ; 68-я Всероссийская научная конференция МФТИ. — URL: <https://conf.mipt.ru/conference/24655> (дата обр. 20.05.2026).
4. Practices in timetabling in higher education institutions: a systematic review / R. A. Oude Vrielink [и др.] // PATAT 2016. — 2017.
5. Welsh, D. J. An upper bound for the chromatic number of a graph and its application to timetabling problems / D. J. Welsh, M. B. Powell // The Computer Journal. — 1967. — Т. 10, № 1. — С. 85—86.
6. A hybrid algorithm for the examination timetabling problem / L. T. Merlot [и др.] // PATAT 2002. — 2002.
7. A time-predefined local search approach to exam timetabling problems / E. Burke [и др.] // IIE Transactions. — 2004. — Т. 36, № 6. — С. 509—528.
8. Di Gaspero, L. Tabu search techniques for examination timetabling / L. Di Gaspero, A. Schaerf // PATAT 2000. — 2000.
9. Petrovic, S. A multiobjective optimisation technique for exam timetabling based on trajectories / S. Petrovic, Y. Bykov // PATAT 2002. — 2002.

10. Dowsland, K. A. Simulated annealing solutions for multi-objective scheduling and timetabling / K. A. Dowsland // *Modern Heuristic Search Methods*. — 1996. — С. 155—166.
11. A survey of search methodologies and automated system development for examination timetabling / R. Qu [и др.] // *Journal of Scheduling*. — 2009. — Т. 12, № 1. — С. 55—89.
12. Brailsford, S. C. Constraint satisfaction problems: Algorithms and applications / S. C. Brailsford, C. N. Potts, B. M. Smith // *European Journal of Operational Research*. — 1999. — Т. 119, № 3. — С. 557—581.
13. Corne, D. Evolutionary timetabling: Practice, prospects and work in progress / D. Corne, P. Ross, H.-L. Fang // *UK planning and scheduling SIG workshop*. — 1994.
14. Burke, E. K. A memetic algorithm for university exam timetabling / E. K. Burke, J. P. Newall, R. F. Weare // *The international conference on the practice and theory of automated timetabling*. — 1995.
15. Dorigo, M. Ant colony optimization theory: A survey / M. Dorigo, C. Blum // *Theoretical Computer Science*. — 2005. — Т. 344, № 2/3. — С. 243—278.
16. Al-Betar, M. A. A harmony search algorithm for university course timetabling / M. A. Al-Betar, A. T. Khader // *Annals of Operations Research*. — 2012. — Т. 194, № 1. — С. 3—31.
17. Российская Федерация. Трудовой кодекс Российской Федерации: ст. 112. Нерабочие праздничные дни / Российская Федерация. — 2001. — Нормативный правовой акт.
18. AloneCoder. Разбираемся с алгоритмом коллапса волновой функции [Электронный ресурс] / AloneCoder. — 2020. — URL: <https://habr.com/ru/companies/vk/articles/497590/> (дата обр. 01.03.2024) ; Текст: электронный.

19. Z3 Theorem Prover [Электронный ресурс] / GitHub. — URL: <https://github.com/z3prover/z3> (дата обр. 24.05.2026).
20. Информация по образовательным программам [Электронный ресурс] / ФГБОУ ВО «МГТУ «СТАНКИН». — URL: https://old.stankin.ru/pages/id_81/page_276 (дата обр. 18.12.2024).
21. Календарный учебный график [Электронный ресурс] / ФГБОУ ВО «МГТУ «СТАНКИН». — URL: https://old.stankin.ru/pages/id_81/page_257 (дата обр. 18.12.2024).
22. Кливанский, Н. Н. Формирование расписания занятий высших учебных заведений / Н. Н. Кливанский // Образовательные ресурсы и технологии. — 2015. — Вып. 9, № 1.
23. Bailly, R. Genetic-WFC: Extending Wave Function Collapse With Genetic Search / R. Bailly, G. Leveux // IEEE Transactions on Games. — 2023. — Т. 15, № 1. — С. 36—45. — DOI: 10.1109/TG.2022.3192722. — URL: <https://ieeexplore.ieee.org/document/9836972> (дата обр. 24.05.2026).
24. SQLite Documentation [Электронный ресурс] / SQLite. — URL: <https://sqlite.org/docs.html> (дата обр. 14.04.2026).
25. Polars user guide [Электронный ресурс] / Polars. — URL: <https://docs.pola.rs/> (дата обр. 14.04.2026).
26. Automatic interactive API documentation [Электронный ресурс] / FastAPI. — URL: <https://fastapi.tiangolo.com/> (дата обр. 14.04.2026).
27. Introduction · Get Started with Nuxt v4 [Электронный ресурс] / Nuxt. — URL: <https://nuxt.com/docs/4.x/getting-started/introduction> (дата обр. 14.04.2026).
28. ebolblga/Grad-Work-Alpha [Электронный ресурс] / GitHub. — URL: <https://github.com/ebolblga/Grad-Work-Alpha> (дата обр. 18.12.2024).

29. Списки групп [Электронный ресурс] / ФГБОУ ВО «МГТУ «СТАНКИН»». — URL: https://old.stankin.ru/pages/id_80/page_242 (дата обр. 18.12.2024).

30. ebolblga/Subject-Parser [Электронный ресурс] / GitHub. — URL: <https://github.com/ebolblga/Subject-Parser> (дата обр. 18.12.2024).

31. Петиция «Расписание за неделю» [Электронный ресурс] / ВКонтакте. — URL: https://vk.com/app7939950?section_type=public_r_app (дата обр. 20.05.2026).

32. Галактика РУЗ — расписание занятий [Электронный ресурс] / Галактика. — URL: <http://galaktika.ru/ruz/> (дата обр. 20.05.2026).

33. 1С:Автоматизированное составление расписания. Университет [Электронный ресурс] / 1С. — URL: https://solutions.1c.ru/asp_univer (дата обр. 20.05.2026).

34. Модуль «Расписание» — ТАНДЕМ.Университет [Электронный ресурс] / ТАНДЕМ. — URL: <https://tandemservice.ru/products/tandem-university/moduli/osnovnye-moduli/raspisanie> (дата обр. 20.05.2026).

35. Экспресс-расписание ВУЗ [Электронный ресурс] / ПрограмБанк. — URL: <https://pbprog.ru/docs/raspisv/> (дата обр. 20.05.2026).